HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

INTELLIGENT SYSTEM DEVELOPMENT

BÁO CÁO BÀI TẬP 03

NEURAL NETWORKS AND LEARNED REPRESENTATION

Từ prepared representation đến biểu diễn ẩn được học
trên dữ liệu bảng, hồi quy giá nhà và văn bản quy mô lớn

Sinh viên: Nguyễn Thành Trung

Mã sinh viên: B23DCCN861

Lớp: D23CTPM01-B

Giảng viên: Dinh Que Tran, Ph.D., Assoc. Prof.

2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU 5
1.1. Mục tiêu và câu hỏi nghiên cứu 5
1.2. Phạm vi báo cáo và nguyên tắc bằng chứng 5
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT.....7
2.1. Multilayer Perceptron và linear layer.....7
2.2. Loss, backpropagation và Adam 7
2.3. Batch, epoch và chế độ train/eval.....7
2.4. Early stopping và model selection 7
2.5. PCA, TruncatedSVD và Linear Probe 8
2.6. Các metric.....8
CHƯƠNG 3. THIẾT KẾ THỰC NGHIỆM CHUNG 9
3.1. Tái lập và cấu hình FULL 9
3.2. Protocol train/validation/test 9
3.3. Training loop chuẩn 10
3.4. Quản lý artifact và standalone source 11
CHƯƠNG 4. BÀI TOÁN DIABETES CLASSIFICATION.....12
4.1. Dataset, target và mất cân bằng lớp 12
4.2. Split và prepared representation.....13
4.3. Traditional ML baselines 15
4.4. Tensor, DataLoader và kiến trúc MLP.....15
4.5. Ba thí nghiệm MLP có kiểm soát.....17
4.6. Final test, confusion matrix và ROC.....18
4.7. Learned representation 19
CHƯƠNG 5. BÀI TOÁN HOUSE PRICE REGRESSION 21
5.1. Dữ liệu, lấy mẫu và phân phối target 21
5.2. Split, preprocessing và log-target.....22
5.3. Inverse log-target và clipping audit.....24

5.4. Traditional regression baselines 26
5.5. MLP regression và ba controlled experiments 27
5.6. Final test và tác động của một extreme prediction 29
5.7. Error analysis theo mức giá và dữ liệu bất thường 31
5.8. Learned representation và kết luận task 32
CHƯƠNG 6. AMAZON REVIEW SENTIMENT CLASSIFICATION 34
6.1. Nguồn dữ liệu, sampling và label mapping 34
6.2. TF-IDF sparse và kiểm soát leakage 35
6.3. Batch-only densification 36
6.4. Traditional ML trên sparse TF-IDF 36
6.5. MLP và quy mô tham số 37
6.6. Final test, confusion matrix và cost-performance 39
6.7. Learned representation 41
6.8. Error analysis theo độ dài và khả năng label anomaly 42
CHƯƠNG 7. SO SÁNH VÀ THẢO LUẬN TỔNG HỢP 44
7.1. Ba mức độ phức tạp của biểu diễn 44
7.2. Độ sâu và learning rate 44
7.3. Learned representation có thực sự mang lại lợi ích? 45
7.4. Metric, threshold và mục tiêu sử dụng 45
7.5. Interpretability và khả năng triển khai 45
CHƯƠNG 8. HẠN CHẾ VÀ ĐE DỌA TỚI TÍNH HỢP LỆ 46
8.1. Hạn chế chung 46
8.2. Diabetes 46
8.3. House Price 46
8.4. Comments 46
8.5. Hướng cải thiện 47
CHƯƠNG 9. KẾT LUẬN 48
CHƯƠNG 10. TÀI LIỆU THAM KHẢO 49
PHỤ LỤC A. SOURCE CODE VÀ ARTIFACT 50

A.1. Standalone source 50
A.2. Saved models, preprocessors và vectorizer 50
A.3. CSV kết quả và figure 50
A.4. Checklist lựa chọn và đánh giá 50

CHƯƠNG 1. GIỚI THIỆU

Assignment 03 khảo sát bước chuyển từ biểu diễn đầu vào đã chuẩn bị sẵn sang biểu diễn ẩn do mạng neural tự học. Ba bài toán được chọn cố ý khác nhau về kiểu dữ liệu và quy mô: phân loại Diabetes trên dữ liệu bảng 21 chiều, hồi quy House Price trên dữ liệu bảng hỗn hợp sau one-hot encoding 61 chiều, và phân loại cảm xúc Amazon Reviews trên TF-IDF thưa 20.000 chiều. Trọng tâm không phải chứng minh Deep Learning luôn tốt hơn, mà kiểm tra có kiểm soát khi nào hidden representation hữu ích và chi phí phải trả là gì.

1.1. Mục tiêu và câu hỏi nghiên cứu

Mỗi bài toán so sánh ba traditional baselines với ba cấu hình MLP: một mạng một hidden layer, một mạng sâu hơn, và cùng mạng sâu với learning rate nhỏ hơn. Mô hình cuối chỉ được chọn trên validation; test giữ kín cho đến sau quyết định. Hai phép kiểm tra learned representation — PCA/SVD và Linear Probe — tách câu hỏi "biểu diễn có cấu trúc tốt hơn không?" khỏi câu hỏi "mô hình dự đoán cuối có thắng không?".

- RQ1: tăng độ sâu từ một lên hai hidden layer có cải thiện nhất quán không?
- RQ2: giảm learning rate từ 10^{-3} xuống 3×10^{-4} có đem lại hội tụ hoặc chất lượng tốt hơn không?
- RQ3: hidden representation có dễ được một linear decoder khai thác hơn prepared input không?
- RQ4: lợi ích metric có tương xứng với parameter count và thời gian CPU hay không?

1.2. Phạm vi báo cáo và nguyên tắc bằng chứng

Báo cáo sử dụng duy nhất output đã lưu trong ba executed notebook và 12 CSV tại results/assignment03. Code listing là trích đoạn trực tiếp từ cell đã chạy; bảng số liệu được dựng lại thành bảng Word native để bảo toàn độ chính xác và khả năng chỉnh sửa. Mười hai figure được chèn trực tiếp từ figures/assignment03. Không có notebook nào được chạy lại, không retrain, và không dùng metric của Assignment 01/02 hay QUICK_MODE.

Kết luận định hướng — Learned representation được đánh giá như một đối tượng khoa học riêng. Một hidden space tốt hơn về Linear Probe không tự động đồng nghĩa final MLP tốt hơn Random Forest hoặc LinearSVC trong hiệu năng thực

dụng.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Multilayer Perceptron và linear layer

Với mini-batch $X \in \mathbb{R}^{Bxd}$, một linear layer tính $Z=XW^T+b$. Trong PyTorch, nn.Linear(d, h) lưu h×d trọng số và h bias; đầu ra có shape [B,h]. Ghép nhiều linear layer mà không có phi tuyến vẫn tương đương một phép biến đổi tuyến tính duy nhất, vì vậy ReLU được đặt sau mỗi hidden linear layer để mô hình hóa quan hệ phi tuyến.

ReLU(z)=max(0,z) giữ gradient đơn giản trên miền dương và tạo activation thưa. Forward propagation lần lượt biến đổi $x \rightarrow h\theta(x) \rightarrow \hat{y}$; $h\theta(x)$ là learned representation được trích ở hidden layer cuối trong các thí nghiệm.

2.2. Loss, backpropagation và Adam

Hai bài toán classification dùng CrossEntropyLoss trực tiếp trên logits. Hàm này kết hợp log-softmax và negative log-likelihood, nên không cần áp softmax trước loss. House Price dùng MSELoss trên target log1p đã standardize; bình phương sai số tạo gradient mạnh cho lỗi lớn, nhưng cũng khiến metric nhạy với outlier. Sau forward, loss.backward() tính gradient theo quy tắc chuỗi; optimizer.step() cập nhật tham số. Adam điều chỉnh learning rate theo moment bậc một và hai của gradient.

2.3. Batch, epoch và chế độ train/eval

Một batch là tập con dùng cho một lần cập nhật; một epoch là một lượt qua toàn bộ train set. model.train() bật hành vi huấn luyện của các layer phụ thuộc chế độ, còn model.eval() cố định hành vi đánh giá. torch.no_grad() tắt lưu đồ thị gradient trong validation/test, giảm memory và tránh cập nhật ngoài ý muốn. Dù các MLP ở đây không dùng dropout hay batch normalization, việc duy trì đúng protocol vẫn làm code an toàn và có thể mở rộng.

2.4. Early stopping và model selection

Early stopping theo dõi validation loss. Mỗi khi loss giảm, state_dict được sao chép làm best_state; nếu không cải thiện liên tiếp đủ patience, vòng lặp dừng và best_state được phục hồi. Validation tham gia chọn kiến trúc/hyperparameter; test chỉ ước lượng generalization sau selection. Quy tắc này ngăn test leakage và tránh báo cáo một kết quả đã được tối ưu ngầm trên test.

2.5. PCA, TruncatedSVD và Linear Probe

PCA chiếu dữ liệu dense lên các hướng phương sai lớn nhất. Với TF-IDF sparse, TruncatedSVD thực hiện phép chiếu tương tự mà không cần center và không phá cấu trúc thưa. Hai phép chiếu 2D chỉ mang tính mô tả: chồng lấn trên hình không phủ định khả năng phân tách ở không gian nhiều chiều. Linear Probe đóng băng representation, huấn luyện một mô hình tuyến tính mới và đo mức độ thông tin mục tiêu có thể được giải mã tuyến tính. Probe tốt hơn là bằng chứng về linear decodability, không phải bằng chứng nhân quả hay bảo đảm deployment superiority.

2.6. Các metric

Classification được đọc bằng Accuracy, Precision, Recall, F1 và ROC-AUC. Với Diabetes mất cân bằng, F1/Recall/AUC quan trọng hơn Accuracy đứng riêng. Regression dùng MAE, RMSE và R^2: MAE phản ánh lỗi tuyệt đối điển hình; RMSE phạt mạnh lỗi cực đoan; R^2 âm nghĩa là mô hình kém hơn dự đoán trung bình trên cùng tập đánh giá. Vì House dùng log-target, mọi metric chính phải được inverse về USD và audit numerical safeguard minh bạch.

CHƯƠNG 3. THIẾT KẾ THỰC NGHIỆM CHUNG

3.1. Tái lập và cấu hình FULL

Cả ba notebook đặt RANDOM_STATE=42, QUICK_MODE=False, NUM_WORKERS=0 và chạy trên CPU. Python, NumPy và PyTorch dùng cùng seed; CUDA cũng được seed khi khả dụng. Optional diagnostics và seed-stability không chạy, vì vậy các kết luận chỉ áp dụng cho một seed/split đã định.

Đoạn mã 3.1. Thiết lập seed và chế độ chạy FULL (trích notebook Diabetes)

```
RANDOM_STATE = 42
QUICK_MODE = False
RUN_OPTIONAL_DIAGNOSTICS = False
RUN_SEED_STABILITY = False

DATA_RELATIVE_PATH = Path("data/diabetes/diabetes_binary_health_indicators_BRFSS2015.csv")
BATCH_SIZE = 512 if not QUICK_MODE else 128
EPOCHS = 25 if not QUICK_MODE else 2
PATIENCE = 5 if not QUICK_MODE else 2
NUM_WORKERS = 0

def set_seed(seed=RANDOM_STATE):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

def find_repo_root(relative_data_path):
    start = Path.cwd().resolve()
    for candidate in [start, *start.parents]:
        if (candidate / relative_data_path).exists():
            return candidate
    raise FileNotFoundError(f"Không tìm thấy {relative_data_path} từ {start}")

# … phần còn lại của cell được lược khỏi báo cáo …
```

Kết quả đầu ra:

```
{'quick_mode': False, 'random_state': 42, 'batch_size': 512, 'epochs': 25, 'patience': 5}
{'device': 'cpu', 'cuda_available': False}
```

Phân tích kết quả: QUICK_MODE=False xác nhận metric thuộc lần chạy FULL. CPU giúp so sánh thời gian trong cùng điều kiện, nhưng thời gian tuyệt đối không đại diện GPU. Seed cố định làm split và khởi tạo có thể tái lập trong môi trường tương thích.

3.2. Protocol train/validation/test

Tiền xử lý luôn fit trên train rồi transform validation/test. Ba MLP dùng cùng split, batch policy, loss và selection rule trong từng task; chỉ hidden_dims hoặc learning_rate thay đổi. Baselines và MLP được so sánh trên validation để lựa chọn, sau đó bảng test chứa traditional models và duy nhất final MLP. Protocol này tạo controlled comparison: khác biệt quan sát được không bị trộn với việc đổi dữ liệu hoặc preprocessing.

Bảng 3.1. Thiết kế chung của ba thí nghiệm

Thành phần	Diabetes	House Price	Comments
Split	70/15/15, stratified	70/15/15	800K/100K/200K official
Prepared input	Scale + passthrough	Impute + scale + OHE	Sparse TF-IDF
Baseline MLP	21→64→2	61→64→1	20000→64→2
Deeper MLP	21→64→32→2	61→64→32→1	20000→64→32→2
Optimizer	Adam	Adam	Adam
Selection	Validation F1, rồi loss	Validation RMSE, rồi MAE	Validation F1, rồi loss
Representation	PCA + Logistic Probe	PCA + Linear Probe	SVD/PCA + Logistic Probe

3.3. Training loop chuẩn

Đoạn mã 3.2. Các bước backpropagation và validation có early stopping

```
def train_mlp_classifier(config, train_loader, val_loader, class_weights):
    model = MLP(X_train.shape[1], config["hidden_dims"], 2).to(device)
    criterion = nn.CrossEntropyLoss(weight=class_weights.to(device))
    optimizer = torch.optim.Adam(model.parameters(), lr=config["learning_rate"])
    best_state, best_loss, best_epoch, wait = copy.deepcopy(model.state_dict()), float("inf"),
0, 0
    history, started = [], time.perf_counter()
    for epoch in range(1, config["epochs"] + 1):
        model.train()
        running_loss, seen = 0.0, 0
        train_labels, train_pred = [], []
        for X_b, y_b in train_loader:
            X_b, y_b = X_b.to(device), y_b.to(device)
            optimizer.zero_grad()
            logits = model(X_b)
            assert X_b.dtype == torch.float32 and y_b.dtype == torch.long
            assert logits.ndim == 2 and logits.shape == (y_b.shape[0], 2)
            assert logits.device == X_b.device == next(model.parameters()).device
            loss = criterion(logits, y_b)
            loss.backward()
            optimizer.step()
            running_loss += loss.item() * len(y_b); seen += len(y_b)
            train_labels.extend(y_b.detach().cpu().numpy());
train_pred.extend(logits.argmax(1).detach().cpu().numpy())
        val_result = evaluate_mlp_classifier(model, val_loader, criterion)
        history.append({"epoch": epoch, "train_loss": running_loss / seen, "train_f1":
f1_score(train_labels, train_pred), "val_loss": val_result["loss"], "val_f1": val_result["f1"],
"val_accuracy": val_result["accuracy"]})
        if val_result["loss"] < best_loss:
            best_loss, best_epoch, wait = val_result["loss"], epoch, 0
            best_state = copy.deepcopy(model.state_dict())
        else:
            wait += 1
            if wait >= config["patience"]: break
    model.load_state_dict(best_state)
```

Giải thích đoạn mã: model.train() được gọi trước vòng mini-batch; zero_grad() xóa gradient tích lũy; logits=model(X_b) thực hiện forward; criterion tính Cross Entropy có class weight; backward tạo gradient; step cập nhật tham số. Cuối epoch, evaluate_mlp_classifier chuyển sang eval/no_grad. best_state được sao chép khi

validation loss giảm và phục hồi sau khi dừng. Việc lưu bản sao state_dict là cần thiết vì giữ tham chiếu trực tiếp sẽ tiếp tục thay đổi cùng model.

3.4. Quản lý artifact và standalone source

Notebook là nguồn code/output chính của báo cáo. Thư mục src/assignment03 gồm common.py, diabetes.py, house_price.py và comments.py, phản ánh lại pipeline dưới dạng module standalone để tái sử dụng; báo cáo không paste toàn bộ các file này. models/assignment03 lưu final_mlp.pt cùng preprocessor hoặc TF-IDF vectorizer; results/assignment03 lưu 12 CSV; figures/assignment03 lưu 12 PNG. Sự tách biệt này làm rõ ba lớp: thực nghiệm có diễn giải trong notebook, logic tái sử dụng trong source, và artifact phục vụ kiểm chứng/deployment.

CHƯƠNG 4. BÀI TOÁN DIABETES CLASSIFICATION

4.1. Dataset, target và mất cân bằng lớp

Dataset BRFSS 2015 có 253.680 quan sát và 22 cột: 21 biến đầu vào cùng target Diabetes_binary. Lớp 0 có 218.334 mẫu (86,0667%), lớp 1 có 35.346 mẫu (13,9333%). Đây là mất cân bằng rõ: một classifier luôn đoán lớp 0 đã có Accuracy khoảng 86% nhưng Recall lớp bệnh bằng 0. Vì vậy pipeline dùng stratification, class weighting và đọc F1/AUC/Recall cùng Accuracy.

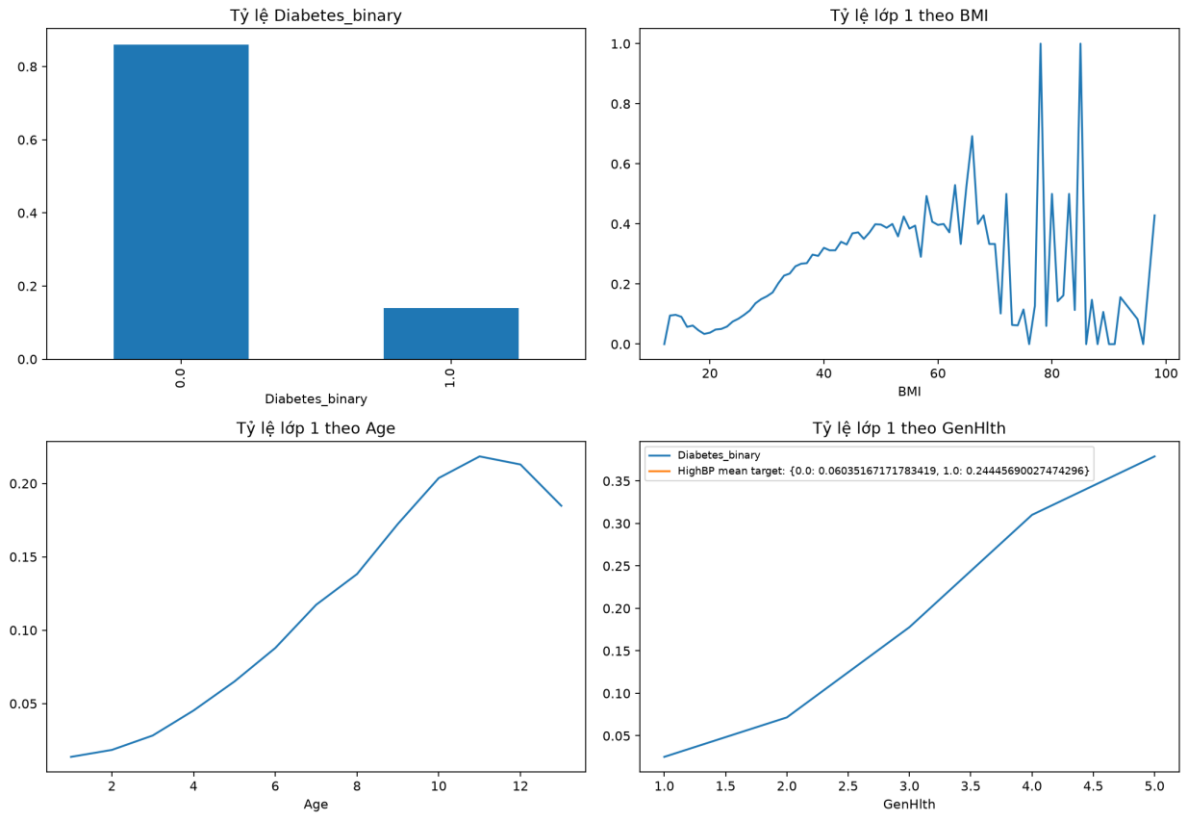
Đoạn mã 4.1. Đọc dữ liệu và kiểm tra schema Diabetes

```
raw_df = pd.read_csv(DATA_PATH, nrows=2_000 if QUICK_MODE else None)
TARGET = "Diabetes_binary"
assert TARGET in raw_df.columns
print("Shape:", raw_df.shape)
display(raw_df.head())
display(raw_df.dtypes.rename("dtype").to_frame())
display(raw_df.describe(include="all").T)
```

Kết quả đầu ra:

```
Shape: (253680, 22)
Target: Diabetes_binary
Class 0: 218334 (0.860667)
Class 1: 35346 (0.139333)
Duplicate rows: 24206
```

Phân tích kết quả: tỷ lệ lớp dương thấp làm Accuracy dễ gây hiểu sai. Con số 24.206 dòng giống hệt chỉ là một tín hiệu chất lượng dữ liệu; với dữ liệu khảo sát rời rạc, nhiều người có thể có cùng vector trả lời, nên không thể kết luận đó là lỗi và notebook không tự động xóa chúng.



Hình 4.1. Quan hệ giữa target Diabetes và các biến sức khỏe

Phân tích hình: Các phân phối cho thấy tỷ lệ Diabetes thay đổi theo những biến như sức khỏe tổng quát, BMI và tuổi. Tuy nhiên đây là quan hệ mô tả trên dữ liệu quan sát, không chứng minh tác động nhân quả. Mất cân bằng lớp vẫn hiện diện trong mọi lát cắt và chi phối lựa chọn metric.

4.2. Split và prepared representation

Split hai bước tạo 70% train, 15% validation và 15% test, đồng thời stratify theo y ở cả hai bước. Bảy biến numeric/ordinal được StandardScaler; 14 biến nhị phân passthrough. ColumnTransformer chỉ fit train, nên mean/standard deviation không nhìn thấy validation/test. Số chiều giữ nguyên d=21.

Đoạn mã 4.2. Chia tập có stratification

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler

binary_features = ["HighBP", "HighChol", "CholCheck", "Smoker", "Stroke",
"HeartDiseaseorAttack", "PhysActivity", "Fruits", "Veggies", "HvyAlcoholConsump",
"AnyHealthcare", "NoDocbcCost", "DiffWalk", "Sex"]
scaled_features = ["BMI", "GenHlth", "MentHlth", "PhysHlth", "Age", "Education", "Income"]
feature_columns = scaled_features + binary_features
X = raw_df[feature_columns]
y = raw_df[TARGET].astype("int64")
X_train_raw, X_temp_raw, y_train, y_temp = train_test_split(X, y, test_size=0.30, stratify=y,
random_state=RANDOM_STATE)
X_val_raw, X_test_raw, y_val, y_test = train_test_split(X_temp_raw, y_temp, test_size=0.50,
stratify=y_temp, random_state=RANDOM_STATE)
for name, frame, labels in [("train", X_train_raw, y_train), ("validation", X_val_raw, y_val),
("test", X_test_raw, y_test)]:
    print(name, frame.shape, labels.value_counts(normalize=True).sort_index().to_dict())
```

Kết quả đầu ra:

```
train (177576, 21) {0: 0.8606681, 1: 0.1393319}
validation (38052, 21) {0: 0.8606644, 1: 0.1393356}
test (38052, 21) {0: 0.8606644, 1: 0.1393356}
```

Phân tích kết quả: ba tỷ lệ gần như trùng nhau, vì vậy chênh lệch metric ít có khả năng đến từ class prior bị lệch giữa split. Test 38.052 mẫu đủ lớn để confusion matrix có ý nghĩa hơn một hold-out nhỏ.

Đoạn mã 4.3. Fit ColumnTransformer trên train và transform các split

```
preprocessor = ColumnTransformer([
    ("scaled", StandardScaler(), scaled_features),
    ("binary", "passthrough", binary_features),
], remainder="drop", verbose_feature_names_out=False)
X_train = preprocessor.fit_transform(X_train_raw).astype("float32")
X_val = preprocessor.transform(X_val_raw).astype("float32")
X_test = preprocessor.transform(X_test_raw).astype("float32")
prepared_feature_names = preprocessor.get_feature_names_out()
print("Prepared shapes:", X_train.shape, X_val.shape, X_test.shape)
display(pd.DataFrame(X_train[:1], columns=prepared_feature_names))
```

Kết quả đầu ra:

```
Prepared shapes: (177576, 21) (38052, 21) (38052, 21)
```

Giải thích đoạn mã: fit_transform chỉ xuất hiện ở X_train_raw; X_val_raw và X_test_raw chỉ đi qua transform. ép float32 tương thích với tham số PyTorch, giảm một nửa memory so với float64. prepared_feature_names giữ liên kết giữa cột transformed và ý nghĩa gốc.

Bảng 4.1. Nhóm biến trong prepared representation Diabetes

Xử lý	Số biến	Biến
StandardScaler	7	BMI, GenHlth, MentHlth, PhysHlth, Age, Education, Income
Passthrough	14	HighBP, HighChol, CholCheck, Smoker, Stroke, HeartDiseaseorAttack, PhysActivity, Fruits, Veggies, HvyAlcoholConsump, AnyHealthcare, NoDocbcCost, DiffWalk, Sex

4.3. Traditional ML baselines

Đoạn mã 4.4. Huấn luyện ba baseline classifier

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier

baseline_models = {
    "Logistic Regression": LogisticRegression(max_iter=1_000, class_weight="balanced",
random_state=RANDOM_STATE),
    "Decision Tree": DecisionTreeClassifier(max_depth=12, min_samples_leaf=20,
class_weight="balanced", random_state=RANDOM_STATE),
    "Random Forest": RandomForestClassifier(n_estimators=30 if QUICK_MODE else 200,
max_depth=18, min_samples_leaf=5, class_weight="balanced_subsample", n_jobs=-1,
random_state=RANDOM_STATE),
}
baseline_rows = []
for name, model in baseline_models.items():
    started = time.perf_counter()
    model.fit(X_train, y_train)
    elapsed = time.perf_counter() - started
    pred = model.predict(X_val)
    row = {"model": name, "split": "validation", **classification_metrics(y_val, pred,
sklearn_score(model, X_val)), "training_seconds": elapsed, "parameter_count": np.nan}
    baseline_rows.append(row)
baseline_val_results = pd.DataFrame(baseline_rows)
# … phần còn lại của cell được lược khỏi báo cáo …
```

Bảng 4.2. Kết quả traditional baselines trên validation

Mô hình	Accuracy	Precision	Recall	F1	ROC-AUC	Thời gian (s)
Logistic Regression	0.7302	0.3095	0.7608	0.4400	0.8215	0.35
Decision Tree	0.7086	0.2910	0.7597	0.4208	0.7977	0.76
Random Forest	0.7822	0.3514	0.6654	0.4599	0.8219	18.42

Phân tích kết quả: Random Forest có F1 validation cao nhất (0,4599) và Accuracy 0,7822, nhưng Recall chỉ 0,6654. Logistic Regression hy sinh Accuracy để nâng Recall lên 0,7608; Decision Tree cũng có Recall tương tự nhưng AUC/F1 thấp hơn. Trade-off cho thấy không tồn tại một mô hình thắng tuyệt đối: nếu chi phí bỏ sót người bệnh cao, Recall có thể quan trọng hơn mức tăng F1 nhỏ của RF.

4.4. Tensor, DataLoader và kiến trúc MLP

Đoạn mã 4.5. Tạo TensorDataset và mini-batch dense

```
def make_dense_loader(X_array, y_array, batch_size, shuffle):
    dataset = TensorDataset(torch.from_numpy(np.asarray(X_array, dtype=np.float32)),
torch.from_numpy(np.asarray(y_array, dtype=np.int64)))
    return DataLoader(dataset, batch_size=batch_size, shuffle=shuffle, num_workers=NUM_WORKERS,
pin_memory=torch.cuda.is_available())

train_loader = make_dense_loader(X_train, y_train.to_numpy(), BATCH_SIZE, True)
val_loader = make_dense_loader(X_val, y_val.to_numpy(), BATCH_SIZE, False)
test_loader = make_dense_loader(X_test, y_test.to_numpy(), BATCH_SIZE, False)
X_batch, y_batch = next(iter(train_loader))
print("X_batch:", X_batch.shape, "y_batch:", y_batch.shape)
```

Kết quả đầu ra:

```
X_batch: torch.Size([512, 21])
y_batch: torch.Size([512])
X dtype: torch.float32; y dtype: torch.int64
```

Phân tích kết quả: CrossEntropyLoss yêu cầu logits [B,2] và nhãn integer [B]. Batch [512,21] khớp input dimension 21; shuffle chỉ bật cho train. pin_memory phụ thuộc CUDA nhưng thiết bị thực tế là CPU.

Đoạn mã 4.6. MLP trả logits và tùy chọn hidden representation

```
class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dims, output_dim):
        super().__init__()
        layers = []
        previous = input_dim
        for width in hidden_dims:
            layers.extend([nn.Linear(previous, width), nn.ReLU()])
            previous = width
        layers.append(nn.Linear(previous, output_dim))
        self.network = nn.Sequential(*layers)
        self.hidden_dims = tuple(hidden_dims)

    def forward(self, x, return_hidden=False):
        hidden = x
        hidden_states = []
        for layer in self.network[:-1]:
            hidden = layer(hidden)
            if isinstance(layer, nn.ReLU):
                hidden_states.append(hidden)
        output = self.network[-1](hidden)
        return (output, hidden_states[-1]) if return_hidden else output

def count_trainable_parameters(model):
    return sum(parameter.numel() for parameter in model.parameters() if
parameter.requires_grad)

def architecture_summary(model, input_dim, output_dim):
    return {
        "input_dim": input_dim,
        "hidden_dims": model.hidden_dims,
# … phần còn lại của cell được lược khỏi báo cáo …
```

Giải thích đoạn mã: vòng lặp sinh nn.Linear + nn.ReLU cho mỗi hidden width, sau đó thêm output layer. Với Baseline, [B,21]→[B,64]→[B,2]; với Deeper, [B,21]→[B,64]→[B,32]→[B,2]. return_hidden=True trả activation sau ReLU cuối để PCA và Linear Probe. Parameter count Baseline là 21×64+64 + 64×2+2 = 1.538; Deeper thêm 64×32+32 và đổi output thành 32×2+2, tổng 3.554.

Kết quả đầu ra:

```
Class counts: [152834 24742]
Class weights: [0.5809440, 3.5885539]
Baseline parameters: 1538
Deeper parameters: 3554
```

Phân tích kết quả: trọng số lớp dương gấp khoảng 6,18 lần lớp âm, khiến loss phạt mạnh hơn khi bỏ sót lớp bệnh. Đây là lý do MLP có Recall cao nhưng Precision thấp; threshold mặc định vẫn là argmax logits.

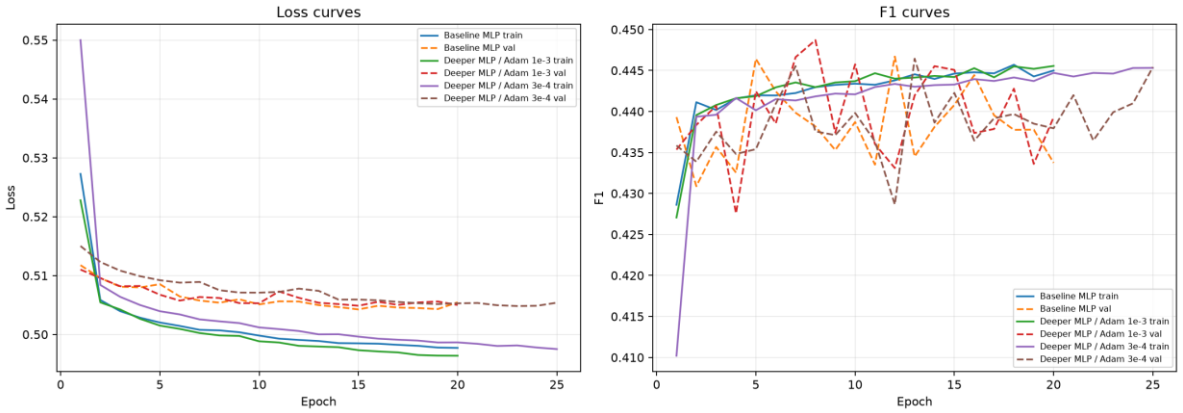
4.5. Ba thí nghiệm MLP có kiểm soát

Đoạn mã 4.7. Cấu hình Baseline, Deeper và learning-rate experiment

```
experiment_configs = [
    {"name": "Baseline MLP", "hidden_dims": [64], "learning_rate": 1e-3, "epochs": EPOCHS,
"patience": PATIENCE},
    {"name": "Deeper MLP / Adam 1e-3", "hidden_dims": [64, 32], "learning_rate": 1e-3,
"epochs": EPOCHS, "patience": PATIENCE},
    {"name": "Deeper MLP / Adam 3e-4", "hidden_dims": [64, 32], "learning_rate": 3e-4,
"epochs": EPOCHS, "patience": PATIENCE},
]
mlp_models, mlp_histories, experiment_rows = {}, {}, []
for config in experiment_configs:
    set_seed()
    model, history, result = train_mlp_classifier(config, train_loader, val_loader,
class_weights)
    mlp_models[config["name"]], mlp_histories[config["name"]] = model, history
    experiment_rows.append({"experiment": config["name"], "architecture":
f"{X_train.shape[1]}→" + "→".join(map(str, config["hidden_dims"])) + "→2", "input_dim":
X_train.shape[1], "output_dim": 2, "hidden_dims": str(config["hidden_dims"]), "optimizer":
"Adam", "learning_rate": config["learning_rate"], "batch_size": BATCH_SIZE, "max_epochs":
EPOCHS, "early_stopping_patience": PATIENCE, "parameter_count":
count_trainable_parameters(model), **result})
mlp_experiment_results = pd.DataFrame(experiment_rows)
display(mlp_experiment_results.sort_values("f1", ascending=False))
```

Bảng 4.3. Kết quả ba MLP trên validation

Thí nghiệm	Kiến trúc	LR	Params	Best epoch	F1	AUC	Loss	Time (s)
Baseline MLP	21→64→2	0.001	1538	15	0.4408	0.8292	0.5043	117.70
Deeper MLP / Adam 1e-3	21→64→32→2	0.001	3554	15	0.4451	0.8288	0.5049	133.97
Deeper MLP / Adam 3e-4	21→64→32→2	0.0003	3554	23	0.4399	0.8287	0.5048	180.53



Hình 4.2. Learning curves của ba MLP Diabetes

Phân tích hình: Baseline đạt best epoch 15 với F1 0,4408. Deeper 10^{-3} cũng best epoch 15 và tăng F1 lên 0,4451, nhưng AUC giảm rất nhẹ. Learning rate 3×10^{-4} cần đến epoch 23 và 180,53 giây nhưng F1 chỉ 0,4399. Vì chỉ learning rate thay đổi, kết quả cho thấy bước học nhỏ hơn hội tụ chậm hơn mà không đem lại lợi ích validation.

Phân tích kết quả: Deeper 10^{-3} được chọn theo validation F1 rồi loss. Mức tăng so với Baseline chỉ 0,00427 F1, trong khi parameter tăng 2,31 lần và thời gian tăng khoảng

16,27 giây. Độ sâu thêm có lợi nhỏ, không phải bước nhảy chất lượng. AUC của ba cấu hình gần nhau cho thấy thứ hạng xác suất ổn định hơn quyết định threshold.

4.6. Final test, confusion matrix và ROC

Đoạn mã 4.8. Chọn final MLP bằng validation trước khi mở test

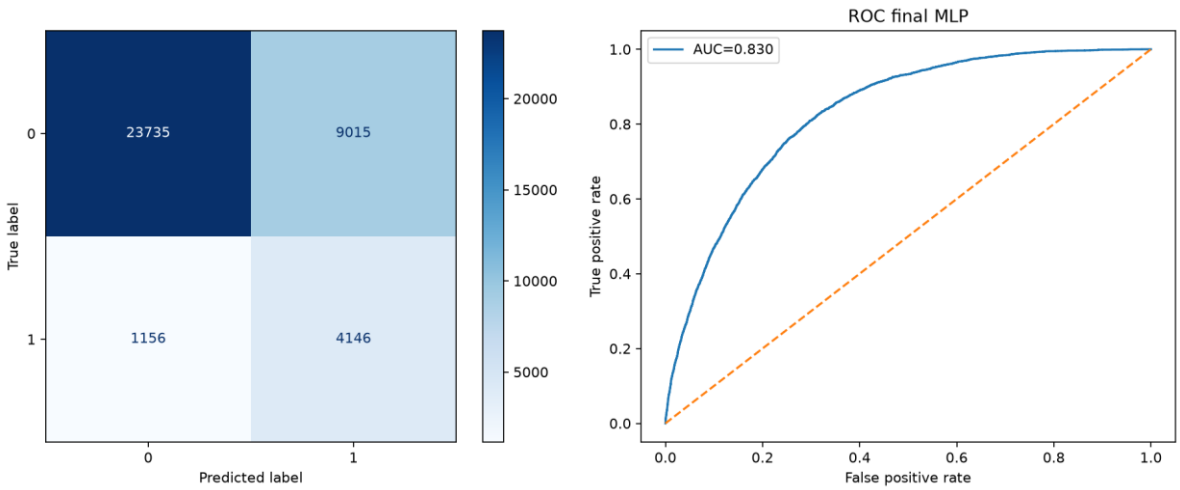
```
best_name = mlp_experiment_results.sort_values(["f1", "loss"], ascending=[False,
True]).iloc[0]["experiment"]
final_mlp = mlp_models[best_name]
final_config = next(item for item in experiment_configs if item["name"] == best_name)
criterion = nn.CrossEntropyLoss(weight=class_weights.to(device))
mlp_test = evaluate_mlp_classifier(final_mlp, test_loader, criterion)
test_rows = []
for name, model in baseline_models.items():
    pred = model.predict(X_test)
    test_rows.append({"model": name, "split": "test", **classification_metrics(y_test, pred,
sklearn_score(model, X_test)), "training_seconds":
baseline_val_results.set_index("model").loc[name, "training_seconds"], "parameter_count":
np.nan, "learned_representation": False})
test_rows.append({"model": best_name, "split": "test", **{k: mlp_test[k] for k in ["accuracy",
"precision", "recall", "f1", "roc_auc"]}, "training_seconds":
mlp_experiment_results.set_index("experiment").loc[best_name, "training_seconds"],
"parameter_count": count_trainable_parameters(final_mlp), "learned_representation": True})
model_comparison = pd.DataFrame(test_rows); model_comparison["input_representation"] = "BRFSS
scaled ordinal/numeric + binary passthrough"; model_comparison["interpretability"] =
np.where(model_comparison["learned_representation"], "Phân tán; cần probe", "Cao hơn với
linear/tree")
display(Markdown(f"**Final MLP được chọn bằng validation:** {best_name}. Test chỉ được mở sau
khi lựa chọn."))
display(model_comparison.sort_values("f1", ascending=False))
```

Bảng 4.4. So sánh bốn mô hình trên test Diabetes

Mô hình	Accuracy	Precision	Recall	F1	ROC-AUC	Params
Logistic Regression	0.733417	0.312819	0.763108	0.443738	0.821556	—
Decision Tree	0.713313	0.295499	0.764051	0.426174	0.800066	—
Random Forest	0.785005	0.353546	0.655413	0.459322	0.824844	—
Deeper MLP / Adam 1e-3	0.732708	0.315022	0.781969	0.449114	0.829583	3,554

Kết quả đầu ra:

```
Final MLP: Deeper MLP / Adam 1e-3
Accuracy=0.732708; Precision=0.315022; Recall=0.781969
F1=0.449114; ROC-AUC=0.829583
Confusion: TN=23735, FP=9015, FN=1156, TP=4146
```



Hình 4.3. Confusion matrix và ROC của final MLP Diabetes

Phân tích hình: MLP phát hiện 4.146 ca dương và bỏ sót 1.156 ca, tương ứng Recall 0,7820. Đổi lại có 9.015 false positive, làm Precision chỉ 0,3150. ROC-AUC 0,8296 cao nhất trong bảng test, nghĩa là khả năng xếp hạng rủi ro tốt; nhưng threshold hiện tại tạo gánh nặng false positive đáng kể.

Phân tích kết quả: Random Forest vẫn thắng F1 test (0,459322 so với 0,449114) và Accuracy, trong khi MLP thắng Recall và AUC. Nếu mục tiêu là screening, MLP có thể phù hợp hơn vì giảm false negative; nếu cần cân bằng precision–recall ở threshold mặc định, RF tốt hơn. Báo cáo vì thế không gọi neural network là winner toàn diện.

4.7. Learned representation

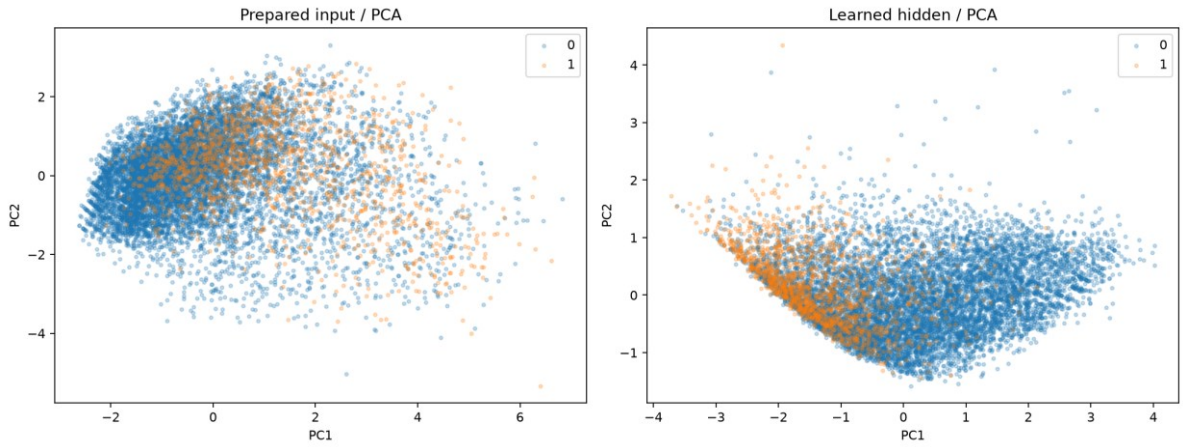
Đoạn mã 4.9. Trích hidden representation đã đóng băng

```
def extract_hidden_dense(model, X_array, batch_size=1024):
    model.eval(); outputs = []
    loader = DataLoader(TensorDataset(torch.from_numpy(np.asarray(X_array, dtype=np.float32))),
batch_size=batch_size, shuffle=False)
    with torch.no_grad():
        for (X_b,) in loader:
            _, hidden = model(X_b.to(device), return_hidden=True)
            outputs.append(hidden.cpu().numpy())
    return np.vstack(outputs)

H_train = extract_hidden_dense(final_mlp, X_train)
H_val = extract_hidden_dense(final_mlp, X_val)
rng = np.random.default_rng(RANDOM_STATE); probe_idx = rng.choice(len(X_test), size=min(10_000,
len(X_test)), replace=False)
H_probe = extract_hidden_dense(final_mlp, X_test[probe_idx])
print("Input/hidden shapes:", X_test[probe_idx].shape, H_probe.shape)
```

Kết quả đầu ra:

```
Input/hidden shapes: (10000, 21) (10000, 32)
```



Hình 4.4. PCA của prepared input và hidden space Diabetes

Phân tích hình: Hidden PCA thay đổi hình học của dữ liệu và làm xu hướng phân tách lớp rõ hơn đôi chút, nhưng hai lớp vẫn chồng lấn mạnh. PCA chỉ giữ hai phương sai lớn nhất nên không nên suy luận trực tiếp rằng mọi thông tin phân loại đều hiện trên hình.

Đoạn mã 4.10. Linear Probe trên prepared input và frozen hidden

```
probe_rows = []
for representation, train_features, val_features in [("Prepared input", X_train, X_val),
("Frozen hidden", H_train, H_val)]:
    probe = LogisticRegression(max_iter=1_000, class_weight="balanced",
random_state=RANDOM_STATE)
    probe.fit(train_features, y_train)
    pred = probe.predict(val_features); score = probe.predict_proba(val_features)[:, 1]
    probe_rows.append({"representation": representation, **classification_metrics(y_val, pred,
score)})
probe_results = pd.DataFrame(probe_rows)
display(probe_results)
```

Bảng 4.5. Linear Probe Diabetes trên validation

Representation	Accuracy	Precision	Recall	F1	ROC-AUC
Prepared input	0.730185	0.309522	0.760845	0.440033	0.821494
Frozen hidden	0.717860	0.304504	0.798189	0.440833	0.828577

Phân tích kết quả: hidden probe tăng AUC 0,00708 và Recall 0,03734, nhưng F1 chỉ tăng 0,00080 và Accuracy giảm. Biểu diễn ẩn chứa tín hiệu xếp hạng dễ tuyến tính hóa hơn, song threshold/class-weight trade-off chưa biến thành cải thiện F1 đáng kể. Kết luận của task là nuanced: learned representation có lợi nhẹ; Random Forest vẫn là mô hình F1 tốt nhất.

CHƯƠNG 5. BÀI TOÁN HOUSE PRICE REGRESSION

5.1. Dữ liệu, lấy mẫu và phân phối target

File USA Real Estate có 2.226.382 dòng và 12 cột. Notebook quét theo chunk để audit thiếu/giá trị không hợp lệ, đồng thời lấy mẫu tái lập 400.000 dòng hợp lệ cho modeling. Sáu feature gồm bed, bath, acre_lot, house_size, state và status; price là target. Lấy mẫu 400K giữ chi phí Random Forest/MLP khả thi trên CPU nhưng đồng thời giới hạn độ bao phủ của full dataset.

Đoạn mã 5.1. Quét CSV theo chunk và tạo modeling sample

```
MODEL_COLUMNS = ["bed", "bath", "acre_lot", "house_size", "state", "status", "price"]
PROFILE_COLUMNS = MODEL_COLUMNS + ["street", "city", "zip_code", "brokered_by"]
rng = np.random.default_rng(RANDOM_STATE)
sampling_fraction = min(1.0, 1.35 * MODEL_SAMPLE_SIZE / EXPECTED_RAW_ROWS)
raw_rows = 0; missing_counts = pd.Series(0, index=PROFILE_COLUMNS, dtype="int64")
invalid_counts = {"price_le_0": 0, "house_size_le_0": 0, "bed_lt_0": 0, "bath_lt_0": 0,
"acre_lot_lt_0": 0}
candidate_parts = []
reader = pd.read_csv(DATA_PATH, usecols=PROFILE_COLUMNS, chunksize=CHUNK_SIZE, nrows=5_000 if
QUICK_MODE else None)
for chunk in reader:
    raw_rows += len(chunk); missing_counts = missing_counts.add(chunk.isna().sum(),
fill_value=0)
    invalid_counts["price_le_0"] += int((chunk["price"] <= 0).sum());
invalid_counts["house_size_le_0"] += int((chunk["house_size"] <= 0).sum())
    invalid_counts["bed_lt_0"] += int((chunk["bed"] < 0).sum()); invalid_counts["bath_lt_0"] +=
int((chunk["bath"] < 0).sum()); invalid_counts["acre_lot_lt_0"] += int((chunk["acre_lot"] <
0).sum())
    keep = rng.random(len(chunk)) < (1.0 if QUICK_MODE else sampling_fraction)
    candidate_parts.append(chunk.loc[keep, MODEL_COLUMNS])
candidate_df = pd.concat(candidate_parts, ignore_index=True)
valid_mask = candidate_df["price"].notna() & candidate_df["price"].gt(0)
valid_mask &= candidate_df["house_size"].isna() | candidate_df["house_size"].gt(0)
for feature in ["bed", "bath", "acre_lot"]:
    valid_mask &= candidate_df[feature].isna() | candidate_df[feature].ge(0)
candidate_df = candidate_df.loc[valid_mask]
model_df = candidate_df.sample(n=min(MODEL_SAMPLE_SIZE, len(candidate_df)),
random_state=RANDOM_STATE).reset_index(drop=True)
print("Actual raw shape:", (raw_rows, len(preview.columns)));
display(missing_counts.rename("missing_count").to_frame()); print("Invalid:", invalid_counts);
print("Modeling sample:", model_df.shape)
```

Kết quả đầu ra:

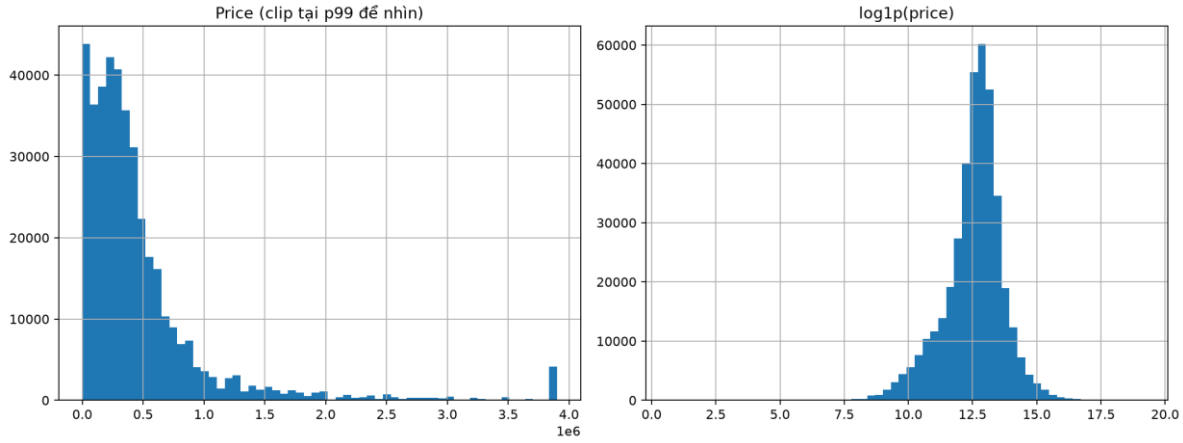
```
Actual raw shape: (2226382, 12)
Modeling sample: (400000, 7)
Selected features: bed, bath, acre_lot, house_size, state, status
Target: price
```

Phân tích kết quả: đọc theo chunk tránh giữ toàn bộ raw frame trong memory; sampling dựa trên RNG có seed làm tập con có thể tái lập. Việc lọc price≤0 và house_size≤0 dựa trên domain validity, còn missing feature được giữ để pipeline impute. Kết quả không đại diện cho một phép census toàn bộ 2,23 triệu dòng.

Bảng 5.1. Tóm tắt phân phối giá

Thống kê	Giá (USD)	Diễn giải

Thống kê	Giá (USD)	Diễn giải
Median (p50)	325,000	Mức giá trung tâm bền vững hơn mean
p99	3,900,000	1% quan sát cao hơn xấp xỉ 3,9 triệu
Maximum	212,500,000	Cực trị lớn gấp khoảng 654 lần median



Hình 5.1. Phân phối price và log1p(price)

Phân tích hình: Price gốc lệch phải rất mạnh; phần lớn mật độ bị nén gần 0 vì vài bất động sản cực đắt. log1p mở rộng vùng giá phổ biến và nén đuôi, tạo target ổn định hơn cho MSE. Phép đổi thang không xóa outlier: khi inverse, sai số USD ở đuôi vẫn rất lớn.

5.2. Split, preprocessing và log-target

Đoạn mã 5.2. Chia 280K/60K/60K và biến đổi target

```
X_raw = model_df.drop(columns=TARGET); y_dollar = model_df[TARGET].astype("float64")
X_train_raw, X_temp_raw, y_train_dollar, y_temp_dollar = train_test_split(X_raw, y_dollar,
test_size=.30, random_state=RANDOM_STATE)
X_val_raw, X_test_raw, y_val_dollar, y_test_dollar = train_test_split(X_temp_raw,
y_temp_dollar, test_size=.50, random_state=RANDOM_STATE)
transform_target = np.log1p if USE_LOG_TARGET else (lambda x: np.asarray(x))
inverse_target = np.expm1 if USE_LOG_TARGET else (lambda x: np.asarray(x))
y_train_model = transform_target(y_train_dollar); y_val_model = transform_target(y_val_dollar);
y_test_model = transform_target(y_test_dollar)
for name, values in [("train", y_train_dollar), ("validation", y_val_dollar), ("test",
y_test_dollar)]: print(name, len(values), values.quantile([.5,.9,.99]).to_dict())
```

Kết quả đầu ra:

```
train 280000 {0.5: 325000.0, 0.9: 949000.0, 0.99: 3900000.0}
validation 60000 {0.5: 325000.0, 0.9: 949000.0, 0.99: 3900000.0}
test 60000 {0.5: 325000.0, 0.9: 949000.0, 0.99: 3900000.0}
```

Phân tích kết quả: ba split có quantile target gần nhau, nhưng random split không bảo đảm cân bằng hoàn toàn các thị trường địa lý hoặc thời gian. USE_LOG_TARGET=True tạo y_model=log1p(price). Tất cả model được fit trên model scale, còn MAE/RMSE/R^2 được báo cáo sau inverse về USD.

Đoạn mã 5.3. Pipeline imputation, scaling và one-hot encoding

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric_features = ["bed", "bath", "acre_lot", "house_size"]
categorical_features = ["state", "status"]
preprocessor = ColumnTransformer([
    ("numeric", Pipeline([("imputer", SimpleImputer(strategy="median")), ("scaler",
StandardScaler())]), numeric_features),
    ("categorical", Pipeline([("imputer", SimpleImputer(strategy="most_frequent")), ("onehot",
OneHotEncoder(handle_unknown="ignore", sparse_output=False, dtype=np.float32))]),
categorical_features),
], verbose_feature_names_out=False)
X_train = preprocessor.fit_transform(X_train_raw).astype("float32")
X_val = preprocessor.transform(X_val_raw).astype("float32"); X_test =
preprocessor.transform(X_test_raw).astype("float32")
prepared_feature_names = preprocessor.get_feature_names_out()
print("Prepared shapes:", X_train.shape, X_val.shape, X_test.shape)
display(pd.DataFrame(X_train[:1], columns=prepared_feature_names))
```

Kết quả đầu ra:

```
Prepared shapes: (280000, 61) (60000, 61) (60000, 61)
Numeric: 4; categorical: 2; final d=61
```

Giải thích đoạn mã: numeric pipeline dùng median imputation rồi StandardScaler; categorical pipeline dùng most-frequent imputation và OneHotEncoder(handle_unknown='ignore'). sparse_output=False tạo dense array vì d=61 nhỏ. Chỉ X_train_raw đi qua fit_transform; validation/test dùng transform, ngăn category/frequency và thống kê numeric từ tương lai rò vào train.

Đoạn mã 5.4. Standardize log-target cho MLP và tạo DataLoader

```
target_mean = float(np.mean(y_train_model)); target_std = float(np.std(y_train_model) + 1e-8)
def standardize_y(values): return ((np.asarray(values) - target_mean) /
target_std).astype("float32").reshape(-1, 1)
def unstandardize_prediction(values): return np.asarray(values).reshape(-1) * target_std +
target_mean
def make_regression_loader(X_array, y_values, shuffle):
    ds = TensorDataset(torch.from_numpy(np.asarray(X_array, dtype=np.float32)),
torch.from_numpy(standardize_y(y_values)))
    return DataLoader(ds, batch_size=BATCH_SIZE, shuffle=shuffle, num_workers=NUM_WORKERS,
pin_memory=torch.cuda.is_available())
train_loader = make_regression_loader(X_train, y_train_model, True); val_loader =
make_regression_loader(X_val, y_val_model, False); test_loader = make_regression_loader(X_test,
y_test_model, False)
X_batch, y_batch = next(iter(train_loader)); print("X_batch:", X_batch.shape, "y_batch:",
y_batch.shape, "target mean/std:", target_mean, target_std)
```

Kết quả đầu ra:

```
X_batch: torch.Size([512, 61])
y_batch: torch.Size([512, 1])
target mean/std: 12.5465359275 / 1.1601016055
```

Phân tích kết quả: MLP tối ưu target z=(log1p(price)−12,5465)/1,1601, giúp output quanh 0 và gradient có scale ổn định. Khi đánh giá, output được unstandardize về log scale rồi expm1 về USD. Baselines dùng log scale không standardize; hai đường khác nhau về optimization nhưng thống nhất metric USD.

5.3. Inverse log-target và clipping audit

Prediction trên log scale có thể ngoại suy đến giá trị khiến expm1 overflow hoặc sinh giá USD phi thực tế. Notebook ưu tiên tính raw metrics nếu inverse raw vẫn finite; song song, nó tạo safeguarded prediction bằng cách thay nonfinite và clip vào min/max log-target chỉ học từ y_train_model. Safeguard là post-processing/domain safeguard, không phải thao tác trung tính: nó có thể thay đổi MAE/RMSE/R^2. Validation/test target tuyệt đối không được dùng để đặt bound.

Đoạn mã 5.5. Inverse prediction với thống kê clipping và raw metrics

```
CLIP_RATE_WARNING_THRESHOLD = 0.01

def _regression_metric_values(y_true_dollar, prediction_dollar, prefix=""):
    mse = mean_squared_error(y_true_dollar, prediction_dollar)
    return {
        f"{prefix}mae": mean_absolute_error(y_true_dollar, prediction_dollar),
        f"{prefix}mse": mse,
        f"{prefix}rmse": np.sqrt(mse),
        f"{prefix}r2": r2_score(y_true_dollar, prediction_dollar),
    }

def inverse_predictions_with_audit(prediction_model_scale, y_true_dollar=None):
    raw_values = np.asarray(prediction_model_scale, dtype=np.float64).reshape(-1)
    train_values = np.asarray(y_train_model, dtype=np.float64).reshape(-1)
    lower, upper = float(np.min(train_values)), float(np.max(train_values))
    finite_mask = np.isfinite(raw_values)

    n_below = int(np.sum(finite_mask & (raw_values < lower)))
    n_above = int(np.sum(finite_mask & (raw_values > upper)))
    n_nonfinite = int(np.sum(~finite_mask))
    n_clipped = n_below + n_above + n_nonfinite
    clip_rate = n_clipped / len(raw_values) if len(raw_values) else 0.0

    # Raw inverse chỉ được tính nếu cả expm1 và tổng squared error an toàn cho float64.
    raw_prediction_dollar = None
    raw_metrics_valid = False
    if len(raw_values) and n_nonfinite == 0:
        metric_safe_limit = np.sqrt(np.finfo(np.float64).max / len(raw_values)) * 0.25
        inverse_safe = (not USE_LOG_TARGET) or bool(np.all(raw_values <=
np.log1p(metric_safe_limit)))
        if inverse_safe:
            with np.errstate(over="ignore", invalid="ignore"):
                candidate = np.asarray(inverse_target(raw_values), dtype=np.float64)
            residual_safe = y_true_dollar is None or bool(
                np.all(np.isfinite(candidate - np.asarray(y_true_dollar, dtype=np.float64)))
                and np.all(np.abs(candidate - np.asarray(y_true_dollar, dtype=np.float64)) <=
metric_safe_limit)
            )
            if np.all(np.isfinite(candidate)) and residual_safe:
                raw_prediction_dollar = candidate
                raw_metrics_valid = True

    # Safeguard dùng duy nhất thống kê train; replacement và clipping đều được audit.
    safeguarded_values = np.nan_to_num(
        raw_values,
        nan=float(np.mean(train_values)),
        posinf=upper,
        neginf=lower,
    )
    safeguarded_values = np.clip(safeguarded_values, lower, upper)
    safeguarded_prediction_dollar = np.maximum(
        0.0,
        np.asarray(inverse_target(safeguarded_values), dtype=np.float64),
    )
    audit = {
        "raw_metrics_valid": raw_metrics_valid,
        "n_below": n_below,
        "n_above": n_above,
        "n_nonfinite": n_nonfinite,
        "n_clipped": n_clipped,
        "clip_rate": clip_rate,
        "clip_lower_train_scale": lower,
        "clip_upper_train_scale": upper,
    }
    return raw_prediction_dollar, safeguarded_prediction_dollar, audit

def regression_metrics(y_true_dollar, prediction_model_scale):
```

Cảnh báo diễn giải — Các metric safeguarded mô tả pipeline sau clipping, không

phản ánh hoàn toàn raw model predictions khi n_clipped>0. Dù clip_rate rất nhỏ, squared error có thể khiến RMSE và R^2 thay đổi mạnh. Vì vậy báo cáo luôn đặt raw và safeguarded cạnh nhau.

5.4. Traditional regression baselines

Đoạn mã 5.6. Huấn luyện Linear, Decision Tree và Random Forest

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor

baseline_models = {
    "Linear Regression": LinearRegression(),
    "Decision Tree Regressor": DecisionTreeRegressor(max_depth=18, min_samples_leaf=10,
random_state=RANDOM_STATE),
    "Random Forest Regressor": RandomForestRegressor(n_estimators=20 if QUICK_MODE else 150,
max_depth=22, min_samples_leaf=5, n_jobs=-1, random_state=RANDOM_STATE),
}
baseline_rows = []
for name, model in baseline_models.items():
    started = time.perf_counter(); model.fit(X_train, y_train_model); elapsed =
time.perf_counter() - started
    baseline_rows.append({"model": name, "split": "validation",
**regression_metrics(y_val_dollar, model.predict(X_val)), "training_seconds": elapsed,
"parameter_count": np.nan})
baseline_val_results = pd.DataFrame(baseline_rows)
display(baseline_val_results.sort_values("rmse"))
report_clipping(baseline_val_results, "Traditional baselines / validation", "model")
```

Bảng 5.2. Traditional baselines trên validation (USD)

Mô hình	MAE safe	RMSE safe	R^2 safe	RMSE raw	R^2 raw	Clip rate	Time (s)
Linear Regression	327,404	2,832,095	-6.0606	20,730,182,359	-378299022.7594	0.01500%	0.78
Decision Tree Regressor	240,720	848,145	0.3668	848,145	0.3668	0.00000%	3.73
Random Forest Regressor	227,886	816,411	0.4133	816,411	0.4133	0.00000%	441.46

Phân tích kết quả: Random Forest thắng validation với RMSE 816.411 USD và R^2 0,4133; Tree đứng sau nhưng train 3,73 giây, tạo trade-off cost-performance hấp dẫn so với 441,46 giây của RF. Linear Regression ngoại suy cực đoan: chỉ 9/60.000 prediction bị clip (0,015%), nhưng raw RMSE tăng lên 20,73 tỷ USD và raw R^2 −378 triệu. Điều này chứng minh tỷ lệ clip nhỏ không đồng nghĩa ảnh hưởng metric nhỏ.

5.5. MLP regression và ba controlled experiments

Đoạn mã 5.7. Kiến trúc MLP cho regression

```
class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dims, output_dim):
        super().__init__()
        layers = []
        previous = input_dim
        for width in hidden_dims:
            layers.extend([nn.Linear(previous, width), nn.ReLU()])
            previous = width
        layers.append(nn.Linear(previous, output_dim))
        self.network = nn.Sequential(*layers)
        self.hidden_dims = tuple(hidden_dims)

    def forward(self, x, return_hidden=False):
        hidden = x
        hidden_states = []
        for layer in self.network[:-1]:
            hidden = layer(hidden)
            if isinstance(layer, nn.ReLU):
                hidden_states.append(hidden)
        output = self.network[-1](hidden)
        return (output, hidden_states[-1]) if return_hidden else output

def count_trainable_parameters(model):
    return sum(parameter.numel() for parameter in model.parameters() if
parameter.requires_grad)

def architecture_summary(model, input_dim, output_dim):
    return {
        "input_dim": input_dim,
        "hidden_dims": model.hidden_dims,
# … phần còn lại của cell được lược khỏi báo cáo …
```

Baseline đi [B,61]→[B,64]→[B,1] với 4.033 tham số. Deeper đi [B,61]→[B,64]→[B,32]→[B,1] với 6.081 tham số. Output không có activation vì standardized log-target có thể âm hoặc dương; thêm ReLU ở output sẽ cắt bỏ phần âm trên standardized scale và làm sai miền bài toán.

Đoạn mã 5.8. Train/evaluate MLP bằng MSELoss

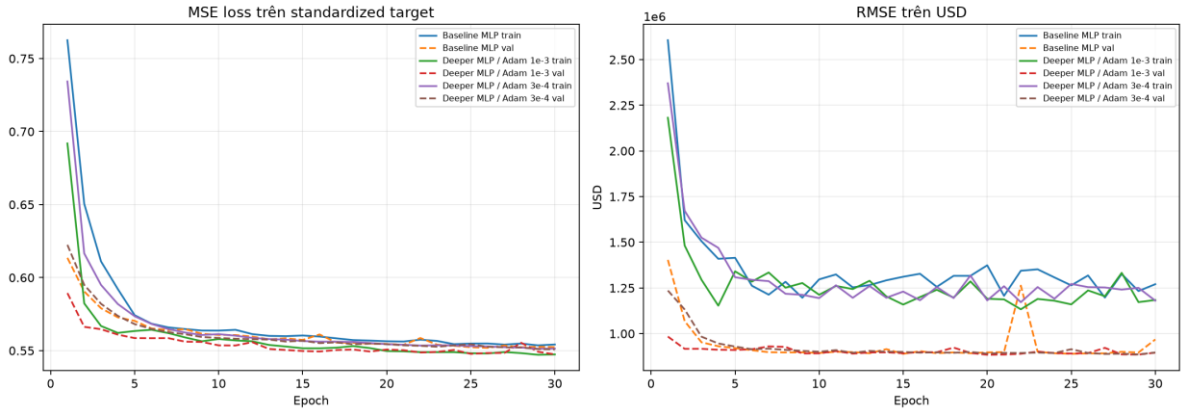
```
def evaluate_mlp_regressor(model, loader, y_true_dollar, criterion):
    model.eval(); losses, predictions = [], []
    with torch.no_grad():
        for X_b, y_b in loader:
            X_b, y_b = X_b.to(device), y_b.to(device); output = model(X_b)
            losses.append(criterion(output, y_b).item() * len(y_b));
predictions.extend(output.cpu().numpy().reshape(-1))
    model_scale = unstandardize_prediction(predictions)
    return {"loss": sum(losses) / len(predictions), **regression_metrics(y_true_dollar,
model_scale)}

def train_mlp_regressor(config):
    model = MLP(X_train.shape[1], config["hidden_dims"], 1).to(device); criterion =
nn.MSELoss(); optimizer = torch.optim.Adam(model.parameters(), lr=config["learning_rate"])
    best_state, best_loss, best_epoch, wait, history = copy.deepcopy(model.state_dict()),
float("inf"), 0, 0, []
    started = time.perf_counter()
    for epoch in range(1, config["epochs"] + 1):
        model.train(); total, seen, train_predictions, train_true_model = 0.0, 0, [], []
        for X_b, y_b in train_loader:
            X_b, y_b = X_b.to(device), y_b.to(device); optimizer.zero_grad(); output =
model(X_b); assert X_b.dtype == torch.float32 and output.shape == y_b.shape; assert
output.device == X_b.device == next(model.parameters()).device; loss = criterion(output, y_b);
loss.backward(); optimizer.step()
            total += loss.item() * len(y_b); seen += len(y_b);
train_predictions.extend(output.detach().cpu().numpy().reshape(-1));
train_true_model.extend(unstandardize_prediction(y_b.detach().cpu().numpy()))
        val_result = evaluate_mlp_regressor(model, val_loader, y_val_dollar, criterion)
        train_rmse = regression_metrics(inverse_target(np.asarray(train_true_model)),
unstandardize_prediction(train_predictions))["rmse"]
        history.append({"epoch": epoch, "train_loss": total/seen, "val_loss":
val_result["loss"], "train_rmse": train_rmse, "val_rmse": val_result["rmse"]})
        if val_result["loss"] < best_loss: best_state, best_loss, best_epoch, wait =
copy.deepcopy(model.state_dict()), val_result["loss"], epoch, 0
        else:
            wait += 1
            if wait >= config["patience"]: break
    model.load_state_dict(best_state); elapsed = time.perf_counter() - started
    return model, pd.DataFrame(history), {"best_epoch": best_epoch, "training_seconds":
elapsed, **evaluate_mlp_regressor(model, val_loader, y_val_dollar, criterion)}
```

Giải thích đoạn mã: evaluate thu output standardized, unstandardize về log scale rồi gọi regression_metrics để nhận cả raw/safeguarded metrics và audit. Trong train, assert kiểm tra dtype, shape và device; early stopping theo validation loss trên standardized scale. History đồng thời lưu RMSE USD để diễn giải.

Bảng 5.3. Kết quả ba MLP House trên validation

Thí nghiệm	Kiến trúc	LR	Params	Best	MAE	RMSE	R^2	Clip
Baseline MLP	61→64→1	0.001	4033	26	259,330	892,011	0.2996	0.0000%
Deeper MLP / Adam 1e-3	61→64→32→1	0.001	6081	30	258,204	895,948	0.2934	0.0000%
Deeper MLP / Adam 3e-4	61→64→32→1	0.0003	6081	30	259,085	896,858	0.2919	0.0000%



Hình 5.2. Learning curves của ba MLP House Price

Phân tích hình: Baseline đạt best epoch 26 và RMSE 892.011 USD. Hai mạng sâu chạy hết 30 epoch nhưng RMSE 895.948 và 896.858, không tốt hơn. Learning rate nhỏ hơn không tạo lợi ích; loss giảm tương đối ổn định nhưng improvement ở cuối nhỏ. Không có prediction validation nào bị clip ở cả ba MLP.

Phân tích kết quả: Baseline được chọn theo RMSE rồi MAE dù Deeper 10^{-3} có MAE thấp hơn khoảng 1.126 USD, vì RMSE của Baseline thấp hơn khoảng 3.937 USD. Đây là quyết định metric-aware: nhiệm vụ ưu tiên kiểm soát lỗi lớn. Mạng sâu hơn tăng 50,8% parameter nhưng không cải thiện generalization.

5.6. Final test và tác động của một extreme prediction

Đoạn mã 5.9. Chọn final MLP và đánh giá bốn model trên test

```
best_name=mlp_experiment_results.sort_values(["rmse","mae"]).iloc[0].experiment;
final_mlp=mlp_models[best_name]; criterion=nn.MSELoss()
mlp_test=evaluate_mlp_regressor(final_mlp,test_loader,y_test_dollar,criterion); test_rows=[]
for name,model in baseline_models.items():
test_rows.append({"model":name,"split":"test",**regression_metrics(y_test_dollar,model.predict(
X_test)),"training_seconds":baseline_val_results.set_index("model").loc[name,"training_seconds"
],"parameter_count":np.nan,"learned_representation":False})
mlp_metric_fields={key:value for key,value in mlp_test.items() if key != "loss"}
test_rows.append({"model":best_name,"split":"test",**mlp_metric_fields,"training_seconds":mlp_e
xperiment_results.set_index("experiment").loc[best_name,"training_seconds"],"parameter_count":c
ount_trainable_parameters(final_mlp),"learned_representation":True})
model_comparison=pd.DataFrame(test_rows); model_comparison["input_representation"]="Scaled
numeric + state/status one-hot";
model_comparison["interpretability"]=np.where(model_comparison.learned_representation,"Phân
tán; cần probe","Linear/tree dễ giải thích hơn"); display(Markdown(f"**Final MLP chọn bằng
validation:** {best_name}")); display(model_comparison.sort_values("rmse"));
report_clipping(model_comparison, "Four-model comparison / test", "model")
```

Bảng 5.4. So sánh final test House: safeguarded và raw

Mô hình	MAE safe	RMSE safe	R^2 safe	MAE raw	RMSE raw	R^2 raw	n clip	Clip rate

Mô hình	MAE safe	RMSE safe	R² safe	MAE raw	RMSE raw	R² raw	n clip	Clip rate
Linear Regression	345,139	3,371,970	-8.670640	97,464,345,236,632,073,662,717,184,312,344,576	23,873,791,394,420,876,940,873,435,890,667,487,232	-484764162171537797928140209226782890172910697285536184639946752.000000	12	0.020000%
Decision Tree Regressor	240,671	864,408	0.364486	240,671	864,408	0.364486	0	0.000000%
Random Forest Regressor	228,400	843,020	0.395546	228,400	843,020	0.395546	0	0.000000%
Baseline MLP	265,721	1,289,163	-0.413524	270,922	2,343,392	-3.670654	1	0.001667%

Kết quả đầu ra:

```
Final MLP = Baseline MLP, 61→64→1, Adam 1e-3, batch 512
Best epoch=26; parameters=4033; training=311.47 s
Safeguarded: MAE=265720.5829; RMSE=1289162.5202; R²=-0.41352358
Raw: MAE=270922.2272; RMSE=2343392.3336; R²=-3.67065432
Audit: n_below=0; n_above=1; n_nonfinite=0; n_clipped=1/60000; clip_rate=0.001667%
Train-only log bounds: [0.6931471806, 19.1744525510]
```

Cảnh báo bắt buộc — Vì final MLP có n_clipped=1, MAE/RMSE/R² safeguarded không phản ánh hoàn toàn raw model predictions. Chỉ 0,001667% prediction bị clip nhưng RMSE giảm từ 2,343 triệu xuống 1,289 triệu USD và R² tăng từ −3,6707 lên −0,4135. Raw metrics vì thế không bị che.

Phân tích kết quả: Random Forest thắng rõ với RMSE 843.020 USD và R² 0,3955; Tree đạt RMSE 864.408 với chi phí train thấp hơn rất nhiều. MLP có MAE 265.721 nhưng đuôi lỗi làm RMSE/R² kém. Linear test còn cực đoan hơn: 12 prediction bị clip (0,02%); raw RMSE khoảng $2{,}387\times10^{37}$. Kết luận model không thay đổi dù dùng safe hay raw: RF tốt nhất.

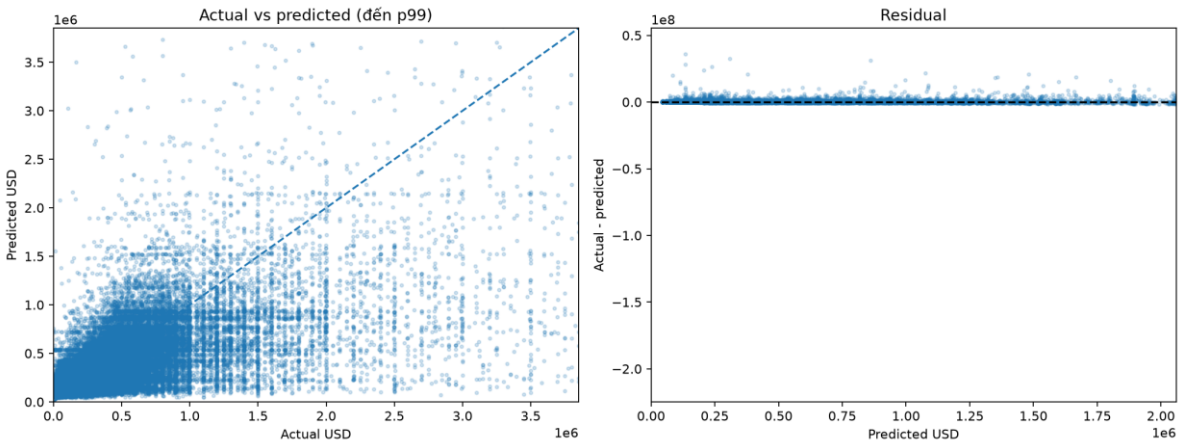
5.7. Error analysis theo mức giá và dữ liệu bất thường

Đoạn mã 5.10. Tạo residual, quintile và top absolute errors

```
final_mlp.eval(); standardized=[]
with torch.no_grad():
    for X_b,_ in test_loader:
standardized.extend(final_mlp(X_b.to(device)).cpu().numpy().reshape(-1))
error_inverse = inverse_predictions_with_audit(unstandardize_prediction(standardized),
y_true_dollar=y_test_dollar)
prediction_dollar = error_inverse[1]
error_audit = pd.DataFrame([{"model": best_name, **error_inverse[2]}])
display(error_audit)
report_clipping(error_audit, "Final MLP error analysis / test", "model")
residual=y_test_dollar.to_numpy()-prediction_dollar
fig,axes=plt.subplots(1,2,figsize=(13,5));
axes[0].scatter(y_test_dollar,prediction_dollar,s=6,alpha=.2);
limit=np.quantile(y_test_dollar,.99); axes[0].plot([0,limit],[0,limit],"--");
axes[0].set(xlim=(0,limit),ylim=(0,limit),title="Actual vs predicted (đến p99)",xlabel="Actual
USD",ylabel="Predicted USD")
axes[1].scatter(prediction_dollar,residual,s=6,alpha=.2);
axes[1].axhline(0,color="black",linestyle="--");
axes[1].set(xlim=(0,np.quantile(prediction_dollar,.99)),title="Residual",xlabel="Predicted
USD",ylabel="Actual - predicted")
plt.tight_layout();
save_figure(FIGURE_DIR/"actual_predicted_residuals.png",dpi=160,bbox_inches="tight");
plt.show()
error_df=X_test_raw.copy(); error_df["actual_price"]=y_test_dollar;
error_df["predicted_price"]=prediction_dollar; error_df["absolute_error"]=np.abs(residual);
error_df["price_range"]=pd.qcut(error_df.actual_price,q=5,duplicates="drop")
display(error_df.groupby("price_range",observed=True).absolute_error.agg(["count","mean","media
n"])); display(error_df.nlargest(15,"absolute_error"))
```

Bảng 5.5. Mean absolute error theo quintile giá test

Nhóm giá từ thấp đến cao	Mean absolute error (USD)	Nhận xét
Q1	95,216	Sai số tương đối vừa phải nhưng lớn so với giá thấp
Q2	87,092	Nhỏ nhất trong năm nhóm
Q3	122,041	Bắt đầu tăng
Q4	205,353	Tăng rõ ở phân khúc cao
Q5	820,261	Đuôi giá chi phối RMSE



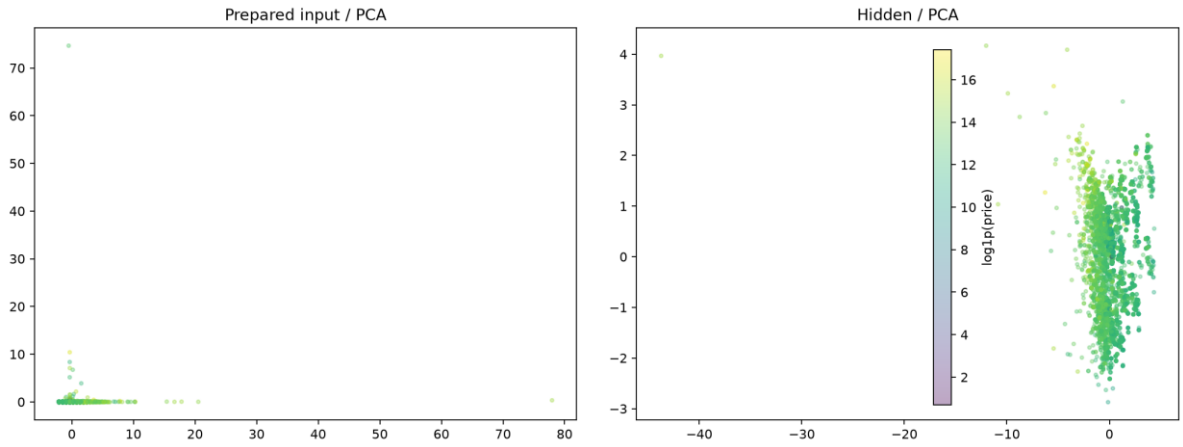
Hình 5.3. Actual–predicted và residual của final MLP House

Phân tích hình: Điểm phân tán xa đường y=x khi giá tăng; residual có phương sai mở rộng theo predicted price. Q5 có mean absolute error 820.261 USD, gấp nhiều lần Q2. Luxury homes thường bị underpredict, nhưng một số record bất thường có bed=444 hoặc bath=222 liên quan extreme error; đây là liên hệ mô tả, không đủ để suy diễn nguyên nhân.

5.8. Learned representation và kết luận task

Đoạn mã 5.11. Trích hidden, PCA và Linear Probe House

```
def extract_hidden_dense(model,X_array,batch_size=1024):
    model.eval(); out=[];
loader=DataLoader(TensorDataset(torch.from_numpy(np.asarray(X_array,dtype=np.float32))),batch_s
ize=batch_size)
    with torch.no_grad():
        for (X_b,) in loader:
out.append(model(X_b.to(device),return_hidden=True)[1].cpu().numpy())
    return np.vstack(out)
H_train=extract_hidden_dense(final_mlp,X_train); H_val=extract_hidden_dense(final_mlp,X_val)
rng=np.random.default_rng(RANDOM_STATE);
idx=rng.choice(len(X_val),min(10_000,len(X_val)),replace=False); H_sample=H_val[idx]
input_2d=PCA(2,random_state=RANDOM_STATE).fit_transform(X_val[idx]);
hidden_2d=PCA(2,random_state=RANDOM_STATE).fit_transform(H_sample);
colors=np.log1p(y_val_dollar.to_numpy()[idx])
fig,axes=plt.subplots(1,2,figsize=(13,5));
s0=axes[0].scatter(input_2d[:,0],input_2d[:,1],c=colors,s=7,alpha=.35,cmap="viridis");
axes[0].set_title("Prepared input / PCA");
axes[1].scatter(hidden_2d[:,0],hidden_2d[:,1],c=colors,s=7,alpha=.35,cmap="viridis");
axes[1].set_title("Hidden / PCA"); fig.colorbar(s0,ax=axes,label="log1p(price)")
plt.tight_layout();
save_figure(FIGURE_DIR/"input_vs_hidden_pca.png",dpi=160,bbox_inches="tight"); plt.show()
```



Hình 5.4. PCA của prepared input và hidden space House

Phân tích hình: Hidden space tái sắp xếp hình học so với prepared input nhưng gradient log-price không tạo thành các dải sạch. Sự chồng lấn phù hợp với target liên tục nhiều nhiễu và feature subset hạn chế; PCA 2D không phản ánh toàn bộ 64 chiều hidden.

Bảng 5.6. Linear Probe House trên validation

Representation	MAE safe	RMSE safe	R² safe	Clip rate

Representation	MAE safe	RMSE safe	R^2 safe	Clip rate
Prepared input	327,404	2,832,095	−6.0606	0.0150%
Frozen hidden	259,461	890,360	0.3022	0%

Phân tích kết quả: linear decoder trên frozen hidden tốt hơn prepared linear model rất rõ, cho thấy MLP đã học nonlinear feature map làm target dễ giải mã tuyến tính. Tuy nhiên final MLP vẫn thua RF/Tree về prediction. Learned representation có giá trị cấu trúc, nhưng không bảo đảm practical superiority; outlier, missingness, sample 400K và feature subset là các giới hạn trọng yếu.

CHƯƠNG 6. AMAZON REVIEW SENTIMENT CLASSIFICATION

6.1. Nguồn dữ liệu, sampling và label mapping

Nguồn cấu hình gồm 3,6 triệu review train và 400.000 review official test. Để thí nghiệm CPU khả thi nhưng vẫn ở quy mô lớn, notebook tạo 800.000 train, 100.000 validation và 200.000 official test. Mỗi split cân bằng chính xác 50/50. Nhãn gốc 1 được map thành Negative=0, nhãn 2 thành Positive=1; cleaning chỉ chuẩn hóa whitespace, không xóa stopword hay stemming.

Đoạn mã 6.1. Mapping nhãn, chuẩn hóa whitespace và hash sampling

```
LABEL_MAP={1:0,2:1}; LABEL_NAME={0:"Negative",1:"Positive"}
def normalize_reviews(series):
    return series.fillna("").astype(str).str.replace(r"\s+"," ",regex=True).str.strip()

def reproducible_csv_subset(csv_path, desired_total, expected_rows):
    if QUICK_MODE:
        frame=pd.read_csv(csv_path,nrows=int(np.ceil(desired_total * 1.10)))
    else:
        threshold=np.uint64(min(2**64-1,int((desired_total*1.25/expected_rows)*(2**64-1))));
parts=[]
        for chunk in pd.read_csv(csv_path,chunksize=CHUNK_SIZE):

hashes=pd.util.hash_pandas_object(chunk[["Review","Label"]],index=False).to_numpy(dtype="uint64
") ^ np.uint64(RANDOM_STATE)
            selected=chunk.loc[hashes<=threshold]
            if len(selected): parts.append(selected)
        frame=pd.concat(parts,ignore_index=True)
    frame=frame.dropna(subset=["Label"]).copy(); frame["label"]=frame["Label"].map(LABEL_MAP);
frame=frame.dropna(subset=["label"]); frame["label"]=frame["label"].astype("int64");
frame["text"]=normalize_reviews(frame["Review"]); frame=frame.loc[frame.text.str.len()>0]
    per_class=desired_total//2;
balanced=pd.concat([group.sample(n=min(per_class,len(group)),random_state=RANDOM_STATE+int(labe
l)) for label,group in frame.groupby("label")],ignore_index=True)
    return balanced.sample(frac=1,random_state=RANDOM_STATE).reset_index(drop=True)
```

Giải thích đoạn mã: hash_pandas_object trên cặp Review/Label, XOR với seed và threshold uint64 chọn candidate theo nội dung thay vì vị trí dòng. Sau khi map nhãn và loại text rỗng, mỗi lớp được sample cùng số lượng rồi shuffle với seed. Cách này tái lập hơn việc lấy n dòng đầu và tránh sample bị lệch class.

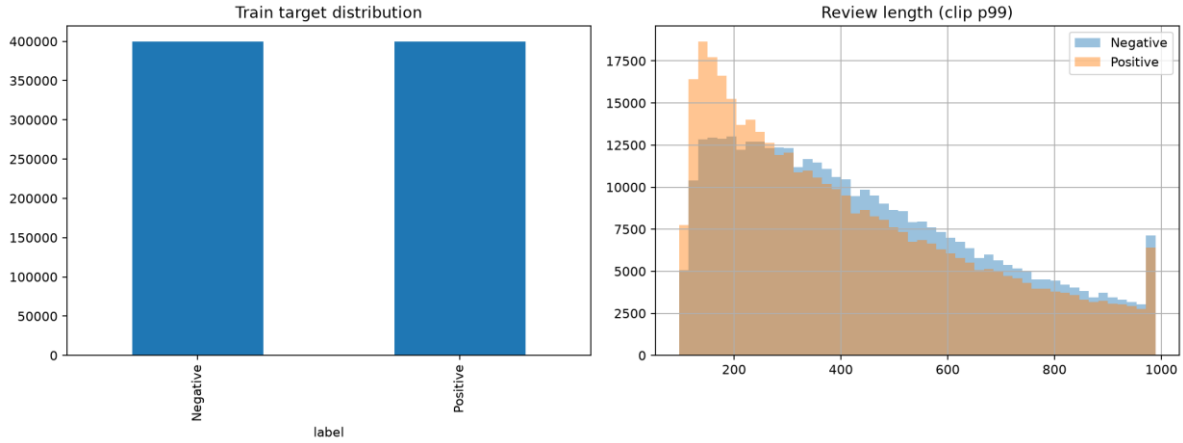
Đoạn mã 6.2. Tạo train/validation và official test

```
train_pool=reproducible_csv_subset(DATA_PATH,COMMENTS_TRAIN_LIMIT+COMMENTS_VAL_LIMIT,EXPECTED_T
RAIN_ROWS)
test_df=reproducible_csv_subset(TEST_PATH,COMMENTS_TEST_LIMIT,EXPECTED_TEST_ROWS)
validation_fraction=COMMENTS_VAL_LIMIT/(COMMENTS_TRAIN_LIMIT+COMMENTS_VAL_LIMIT)
train_df,val_df=train_test_split(train_pool,test_size=validation_fraction,stratify=train_pool.l
abel,random_state=RANDOM_STATE)
train_df=train_df.reset_index(drop=True); val_df=val_df.reset_index(drop=True)
for name,frame in [("train",train_df),("validation",val_df),("official test",test_df)]:
print(name,frame.shape,frame.label.value_counts(normalize=True).sort_index().to_dict(),frame.te
xt.str.len().describe()[["mean","50%","max"]].to_dict())
```

Kết quả đầu ra:

```
train (800000, 4) {0: 0.5, 1: 0.5}
validation (100000, 4) {0: 0.5, 1: 0.5}
official test (200000, 4) {0: 0.5, 1: 0.5}
Raw configured: train=3600000; official test=400000
```

Phân tích kết quả: cân bằng nhân tạo làm Accuracy/F1 dễ đọc và tránh majority baseline chi phối, nhưng phân phối này có thể khác production prevalence. Test vẫn lấy từ file official riêng, nên độc lập nguồn với train pool. Modeling subset không sử dụng toàn bộ 4 triệu review.



Hình 6.1. Phân phối target và độ dài review

Phân tích hình: Hai lớp cân bằng đúng theo thiết kế. Độ dài review lệch phải; review dài tạo nhiều token/bigram và có thể chứa nhiều mệnh đề trái chiều. EDA này dẫn tới error analysis theo quintile độ dài ở phần final test.

6.2. TF-IDF sparse và kiểm soát leakage

Đoạn mã 6.3. Fit TF-IDF train-only và đo mật độ/memory

```
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer=TfidfVectorizer(max_features=TFIDF_MAX_FEATURES,ngram_range=(1,2),sublinear_tf=True,
min_df=TFIDF_MIN_DF,dtype=np.float32,strip_accents="unicode")
X_train=vectorizer.fit_transform(train_df.text); X_val=vectorizer.transform(val_df.text);
X_test=vectorizer.transform(test_df.text)
y_train=train_df.label.to_numpy(); y_val=val_df.label.to_numpy();
y_test=test_df.label.to_numpy(); d=X_train.shape[1]
def sparse_mb(matrix): return
(matrix.data.nbytes+matrix.indices.nbytes+matrix.indptr.nbytes)/1024**2
print("Shapes:",X_train.shape,X_val.shape,X_test.shape,"vocabulary d:",d); print("Train
density:",X_train.nnz/(X_train.shape[0]*X_train.shape[1]),"sparse MB:",sparse_mb(X_train));
display(pd.Series(vectorizer.get_feature_names_out()[:20],name="vocabulary sample"))
```

Kết quả đầu ra:

```
Shapes: (800000, 20000) (100000, 20000) (200000, 20000)
vocabulary d: 20000
Train density: 0.00411180725 (0.411181%)
CSR memory: 504.981 MiB
```

Phân tích kết quả: chỉ khoảng 0,411% ô khác 0, nên CSR là điều kiện cần để giữ memory hợp lý. 800.000×20.000 float32 dense riêng phần values đã xấp xỉ 59,6 GiB, chưa kể overhead; notebook không tạo ma trận dense toàn cục. vectorizer.fit_transform chỉ dùng train; vocabulary/IDF không nhìn validation/test.

TfidfVectorizer dùng max_features=20.000, ngram_range=(1,2), sublinear_tf=True, min_df=5, strip_accents='unicode' và dtype=float32. Unigram mang

tín hiệu từ đơn; bigram giữ một phần phủ định/cụm từ như “not good”. sublinear_tf giảm ảnh hưởng việc lặp token nhiều lần; min_df loại term quá hiếm. Dù vậy TF-IDF là bag-of-ngrams, không mô hình hóa trật tự dài hay ngữ cảnh sâu.

6.3. Batch-only densification

Đoạn mã 6.4. Dataset theo chỉ số và collate CSR→dense từng batch

```
class SparseIndexDataset(Dataset):
    def __init__(self,n_rows): self.n_rows=n_rows
    def __len__(self): return self.n_rows
    def __getitem__(self,index): return index

def make_sparse_loader(matrix,labels,shuffle):
    labels=np.asarray(labels,dtype=np.int64)
    def collate(indices):
        idx=np.asarray(indices,dtype=np.int64);
X_batch=matrix[idx].toarray().astype(np.float32,copy=False); return
torch.from_numpy(X_batch),torch.from_numpy(labels[idx])
    return
DataLoader(SparseIndexDataset(matrix.shape[0]),batch_size=BATCH_SIZE,shuffle=shuffle,num_worker
s=NUM_WORKERS,collate_fn=collate,pin_memory=torch.cuda.is_available())
train_loader=make_sparse_loader(X_train,y_train,True);
train_eval_loader=make_sparse_loader(X_train,y_train,False);
val_loader=make_sparse_loader(X_val,y_val,False);
test_loader=make_sparse_loader(X_test,y_test,False)
X_batch,y_batch=next(iter(train_loader)); print("Batch:",X_batch.shape,y_batch.shape,"dense
batch MB:",X_batch.nelement()*X_batch.element_size()/1024**2)
```

Kết quả đầu ra:

```
Batch: torch.Size([256, 20000]) torch.Size([256])
Dense batch memory: 19.53125 MiB
```

Phân tích kết quả: Dataset chỉ trả index; collate lấy 256 hàng CSR rồi .toarray() thành float32 tensor. Như vậy MLP vẫn nhận dense [B,20000], nhưng peak memory cho input batch khoảng 19,53 MiB thay vì dense toàn bộ corpus. shuffle thay đổi index order của train mà không sao chép toàn ma trận. Đây là cầu nối kỹ thuật quan trọng giữa sparse scikit-learn và dense nn.Linear.

6.4. Traditional ML trên sparse TF-IDF

Đoạn mã 6.5. Huấn luyện Logistic Regression, LinearSVC và ComplementNB

```
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import ComplementNB
from sklearn.svm import LinearSVC
baseline_models={"Logistic
Regression":LogisticRegression(max_iter=300,C=2.0,random_state=RANDOM_STATE,n_jobs=-
1),"LinearSVC":LinearSVC(C=1.0,random_state=RANDOM_STATE),"ComplementNB":ComplementNB(alpha=.5)
}
rows=[]
for name,model in baseline_models.items():
    started=time.perf_counter(); model.fit(X_train,y_train); elapsed=time.perf_counter()-
started; pred=model.predict(X_val)

rows.append({"model":name,"split":"validation",**classification_metrics(y_val,pred,sklearn_scor
e(model,X_val)),"training_seconds":elapsed,"parameter_count":np.nan})
baseline_val_results=pd.DataFrame(rows);
display(baseline_val_results.sort_values("f1",ascending=False))
```

Bảng 6.1. Traditional baselines trên validation Comments

Mô hình	Accuracy	Precision	Recall	F1	ROC-AUC	Time (s)
Logistic Regression	0.93090	0.932804	0.92870	0.930748	0.979711	19.932
LinearSVC	0.93129	0.932284	0.93014	0.931211	0.979861	42.209
ComplementNB	0.88740	0.890713	0.88316	0.886921	0.955692	0.376

Phân tích kết quả: LinearSVC dẫn F1 0,931211 và AUC 0,979861; Logistic gần như ngang bằng nhưng nhanh hơn khoảng 2,12 lần. ComplementNB chỉ mất 0,376 giây song F1 thấp hơn khoảng 0,0443. Với TF-IDF high-dimensional, linear decision boundary đã rất mạnh; đây là chuẩn khó cho MLP vượt qua.

6.5. MLP và quy mô tham số

Đoạn mã 6.6. MLP 20.000 chiều với hidden output

```
class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dims, output_dim):
        super().__init__()
        layers = []
        previous = input_dim
        for width in hidden_dims:
            layers.extend([nn.Linear(previous, width), nn.ReLU()])
            previous = width
        layers.append(nn.Linear(previous, output_dim))
        self.network = nn.Sequential(*layers)
        self.hidden_dims = tuple(hidden_dims)

    def forward(self, x, return_hidden=False):
        hidden = x
        hidden_states = []
        for layer in self.network[:-1]:
            hidden = layer(hidden)
            if isinstance(layer, nn.ReLU):
                hidden_states.append(hidden)
        output = self.network[-1](hidden)
        return (output, hidden_states[-1]) if return_hidden else output

def count_trainable_parameters(model):
    return sum(parameter.numel() for parameter in model.parameters() if
parameter.requires_grad)

def architecture_summary(model, input_dim, output_dim):
    return {
        "input_dim": input_dim,
        "hidden_dims": model.hidden_dims,
# … phần còn lại của cell được lược khỏi báo cáo …
```

Baseline [B,20000]→[B,64]→[B,2] có 1.280.194 tham số; riêng first layer có 20.000×64=1.280.000 weights. Deeper thêm layer 64→32 và output 32→2, tổng 1.282.210 tham số. Vì input dimension thống trị, thêm hidden layer chỉ tăng 2.016 tham số (0,157%), nhưng mỗi epoch vẫn phải dense hóa và nhân hàng trăm nghìn batch-vector với ma trận đầu vào lớn.

Đoạn mã 6.7. Training loop classification cho sparse mini-batches

```
def evaluate_mlp_classifier(model,loader,criterion):
    model.eval(); losses=[]; labels=[]; predictions=[]; scores=[]
    with torch.no_grad():
        for X_b,y_b in loader:
            X_b,y_b=X_b.to(device),y_b.to(device); logits=model(X_b);
losses.append(criterion(logits,y_b).item()*len(y_b)); labels.extend(y_b.cpu().numpy());
predictions.extend(logits.argmax(1).cpu().numpy());
scores.extend(torch.softmax(logits,1)[:,1].cpu().numpy())

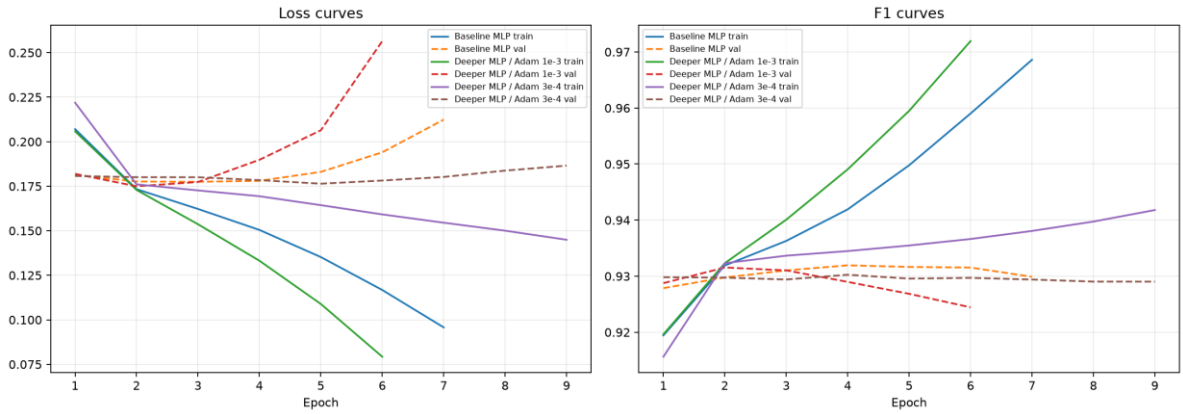
result=classification_metrics(np.asarray(labels),np.asarray(predictions),np.asarray(scores));
result["loss"]=sum(losses)/len(labels); return result

def train_mlp_classifier(config):
    model=MLP(d,config["hidden_dims"],2).to(device); criterion=nn.CrossEntropyLoss();
optimizer=torch.optim.Adam(model.parameters(),lr=config["learning_rate"]);
best_state=copy.deepcopy(model.state_dict()); best_loss=float("inf"); best_epoch=0; wait=0;
history=[]; started=time.perf_counter()
    for epoch in range(1,config["epochs"]+1):
        model.train(); total=seen=0; train_true=[]; train_pred=[]
        for X_b,y_b in train_loader:
            X_b,y_b=X_b.to(device),y_b.to(device); optimizer.zero_grad(); logits=model(X_b);
assert X_b.dtype == torch.float32 and y_b.dtype == torch.long; assert logits.ndim == 2 and
logits.shape == (y_b.shape[0],2); assert logits.device == X_b.device ==
next(model.parameters()).device; loss=criterion(logits,y_b); loss.backward(); optimizer.step();
total+=loss.item()*len(y_b); seen+=len(y_b); train_true.extend(y_b.cpu().numpy());
train_pred.extend(logits.argmax(1).detach().cpu().numpy())
        val=evaluate_mlp_classifier(model,val_loader,criterion);
history.append({"epoch":epoch,"train_loss":total/seen,"val_loss":val["loss"],"train_f1":f1_scor
e(train_true,train_pred),"val_f1":val["f1"],"val_accuracy":val["accuracy"]})
        if val["loss"]<best_loss: best_state=copy.deepcopy(model.state_dict());
best_loss=val["loss"]; best_epoch=epoch; wait=0
        else:
            wait+=1
            if wait>=config["patience"]: break
    model.load_state_dict(best_state); elapsed=time.perf_counter()-started; return
model,pd.DataFrame(history),{"best_epoch":best_epoch,"training_seconds":elapsed,**evaluate_mlp_
classifier(model,val_loader,criterion)}
```

Giải thích đoạn mã: collate đã tạo float32, nhãn long; các assert kiểm tra [B,2] logits và cùng device. CrossEntropyLoss không class-weight vì sample cân bằng. Validation chạy qua 100.000 review mỗi epoch; early stopping theo val loss, không theo F1, làm selection checkpoint ít bị dao động threshold.

Bảng 6.2. Kết quả ba MLP Comments trên validation

Thí nghiệm	Kiến trúc	LR	Params	Best	Accuracy	F1	AUC	Time (s)
Baseline MLP	20000→64→2	0.001	1,280,194	3	0.93100	0.931036	0.980658	1069.26
Deeper MLP / Adam 1e-3	20000→64→32→2	0.001	1,282,210	2	0.93150	0.931553	0.981070	1070.94
Deeper MLP / Adam 3e-4	20000→64→32→2	0.0003	1,282,210	5	0.93014	0.929574	0.980751	1252.30



Hình 6.2. Learning curves của ba MLP Comments

Phân tích hình: Baseline có best validation loss ở epoch 3; Deeper 10^{-3} ở epoch 2; Deeper 3×10^{-4} ở epoch 5. Sau mốc tốt nhất, train loss tiếp tục giảm trong khi validation loss tăng, dù validation F1 có thể dao động nhẹ: đây là dấu hiệu overfitting thực tế. Early stopping phục hồi checkpoint tốt nhất thay vì dùng epoch cuối.

Phân tích kết quả: Deeper 10^{-3} được chọn với F1 0,931553 và AUC 0,981070. So với Baseline, F1 chỉ tăng 0,000518; learning rate nhỏ hơn cần 1.252,30 giây nhưng F1 giảm còn 0,929574. Độ sâu mang lợi ích rất nhỏ; LR nhỏ làm best epoch muộn hơn nhưng không cải thiện generalization.

6.6. Final test, confusion matrix và cost-performance

Đoạn mã 6.8. Chọn final MLP bằng validation và mở official test

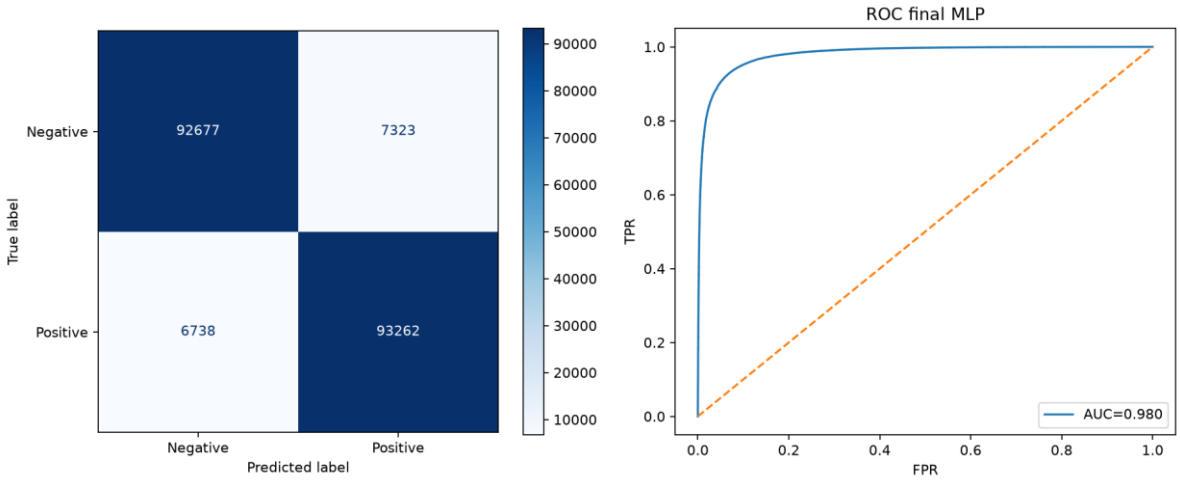
```
best_name=mlp_experiment_results.sort_values(["f1","loss"],ascending=[False,True]).iloc[0].expe
riment; final_mlp=mlp_models[best_name];
mlp_test=evaluate_mlp_classifier(final_mlp,test_loader,nn.CrossEntropyLoss()); test_rows=[]
for name,model in baseline_models.items(): pred=model.predict(X_test);
test_rows.append({"model":name,"split":"official
test","accuracy":accuracy_score(y_test,pred),"precision":precision_score(y_test,pred),"recall":
recall_score(y_test,pred),"f1":f1_score(y_test,pred),"roc_auc":roc_auc_score(y_test,sklearn_sco
re(model,X_test)),"training_seconds":baseline_val_results.set_index("model").loc[name,"training
_seconds"],"parameter_count":np.nan,"learned_representation":False})
test_rows.append({"model":best_name,"split":"official test",**{k:mlp_test[k] for k in
["accuracy","precision","recall","f1","roc_auc"]},"training_seconds":mlp_experiment_results.set
_index("experiment").loc[best_name,"training_seconds"],"parameter_count":count_trainable_parame
ters(final_mlp),"learned_representation":True})
model_comparison=pd.DataFrame(test_rows); model_comparison["input_representation"]="Sparse TF-
IDF unigram+bigram";
model_comparison["interpretability"]=np.where(model_comparison.learned_representation,"Phân
tán; cần probe","Coefficient/token likelihood"); display(Markdown(f"**Final MLP chọn bằng
validation:** {best_name}")); display(model_comparison.sort_values("f1",ascending=False))
```

Bảng 6.3. So sánh bốn mô hình trên official test

Mô hình	Accuracy	Precision	Recall	F1	ROC-AUC	Train (s)	Params
Logistic Regression	0.928780	0.928369	0.929260	0.928814	0.978784	19.93	—
LinearSVC	0.929135	0.928343	0.930060	0.929200	0.979002	42.21	—
ComplementNB	0.884960	0.887486	0.881700	0.884584	0.954376	0.38	—
Deeper MLP / Adam 1e-3	0.929695	0.927196	0.932620	0.929900	0.980112	1070.94	1,282,210

Kết quả đầu ra:

```
Final MLP: Deeper MLP / Adam 1e-3
Accuracy=0.929695; Precision=0.927196; Recall=0.932620
F1=0.929900; ROC-AUC=0.980112
Confusion: TN=92677, FP=7323, FN=6738, TP=93262
LinearSVC test F1=0.929200; MLP advantage=0.000700
MLP/LinearSVC training-time ratio≈25.4×
```



Hình 6.3. Confusion matrix và ROC của final MLP Comments

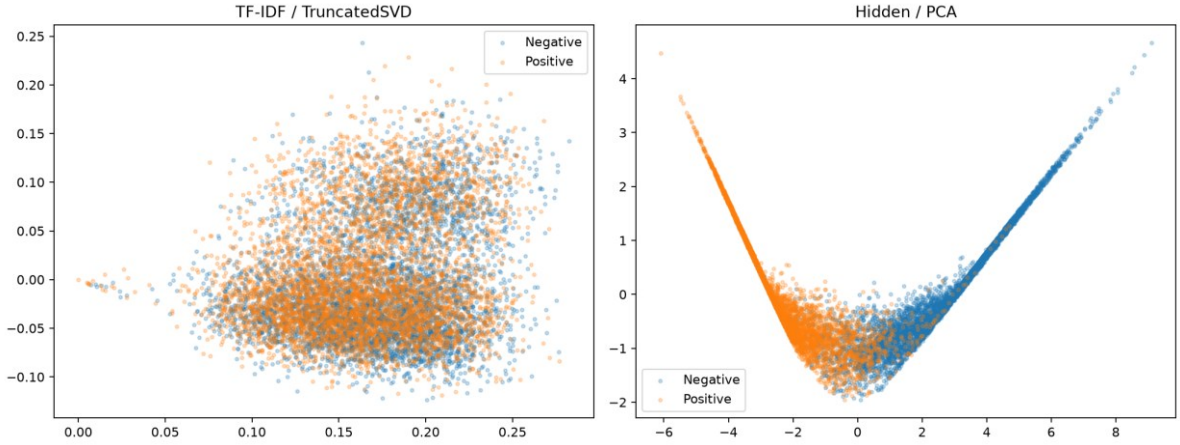
Phân tích hình: Sai số khá cân bằng: 7.323 false positive và 6.738 false negative trên 200.000 review. Recall dương 0,9326 cao hơn Precision 0,9272 một chút. AUC 0,9801 thể hiện ranking rất mạnh, nhưng MLP chỉ hơn LinearSVC khoảng 0,000700 F1 với thời gian train gấp khoảng 25,4 lần.

Phân tích kết quả: MLP là winner metric nhưng biên thắng nhỏ hơn một phần nghìn F1. Với resource hạn chế hoặc cần retrain thường xuyên, LinearSVC là lựa chọn cost-performance thuyết phục; với mục tiêu tối đa hóa AUC/F1 và có đủ compute, MLP có lợi nhỏ. Kết luận này quan trọng hơn tuyên bố đơn giản “Deep Learning thắng”.

6.7. Learned representation

Đoạn mã 6.9. TruncatedSVD của TF-IDF và PCA của hidden space

```
def extract_hidden_sparse(model,loader):
    model.eval(); out=[]
    with torch.no_grad():
        for X_b,_ in loader:
out.append(model(X_b.to(device),return_hidden=True)[1].cpu().numpy())
    return np.vstack(out)
H_train=extract_hidden_sparse(final_mlp,train_eval_loader);
H_val=extract_hidden_sparse(final_mlp,val_loader)
rng=np.random.default_rng(RANDOM_STATE);
idx=rng.choice(X_test.shape[0],min(10_000,X_test.shape[0]),replace=False); X_probe=X_test[idx];
probe_loader=make_sparse_loader(X_probe,y_test[idx],False);
H_probe=extract_hidden_sparse(final_mlp,probe_loader)
input_2d=TruncatedSVD(2,random_state=RANDOM_STATE).fit_transform(X_probe);
hidden_2d=PCA(2,random_state=RANDOM_STATE).fit_transform(H_probe);
fig,axes=plt.subplots(1,2,figsize=(13,5))
for label,color in [(0,"tab:blue"),(1,"tab:orange")]:
    mask=y_test[idx]==label;
axes[0].scatter(input_2d[mask,0],input_2d[mask,1],s=7,alpha=.25,c=color,label=LABEL_NAME[label]
);
axes[1].scatter(hidden_2d[mask,0],hidden_2d[mask,1],s=7,alpha=.25,c=color,label=LABEL_NAME[labe
l])
axes[0].set_title("TF-IDF / TruncatedSVD"); axes[1].set_title("Hidden / PCA");
for ax in axes: ax.legend()
plt.tight_layout();
save_figure(FIGURE_DIR/"tfidf_svd_vs_hidden_pca.png",dpi=160,bbox_inches="tight"); plt.show()
```



Hình 6.4. TF-IDF/TruncatedSVD so với hidden/PCA

Phân tích hình: Hai lớp chồng lấn mạnh trong projection TF-IDF. Hidden PCA tạo cấu trúc dạng chữ V và phân tách trực quan hơn, dù vẫn có vùng giao nhau. Vì SVD và PCA tối ưu phương sai chứ không tối ưu nhãn, hình chỉ là bằng chứng bổ trợ; Linear Probe mới định lượng decodability.

Đoạn mã 6.10. Linear Probe trên TF-IDF và frozen hidden

```
probe_rows=[]
for representation,A,B in [("Prepared TF-IDF",X_train,X_val),("Frozen hidden",H_train,H_val)]:
    probe=LogisticRegression(max_iter=300,random_state=RANDOM_STATE,n_jobs=-1).fit(A,y_train);
prediction=probe.predict(B); score_probe=probe.predict_proba(B)[:,1];
probe_rows.append({"representation":representation,**classification_metrics(y_val,prediction,sc
ore_probe)})
probe_results=pd.DataFrame(probe_rows); display(probe_results)
```

Bảng 6.4. Linear Probe Comments trên validation

Representation	Accuracy	Precision	Recall	F1	ROC-AUC

Representation	Accuracy	Precision	Recall	F1	ROC-AUC
Prepared TF-IDF	0.929200	0.930630	0.927540	0.929082	0.979052
Frozen hidden	0.931420	0.936430	0.925680	0.931024	0.980468

Phân tích kết quả: hidden probe tăng F1 khoảng 0,00194 và AUC 0,00142, đồng thời tăng Precision nhưng giảm Recall. Mạng đã tái mã hóa TF-IDF theo hướng dễ phân tách tuyến tính hơn một chút. Cải thiện này có thật trong split hiện tại nhưng nhỏ, chưa đủ bù computational cost trong mọi bối cảnh.

6.8. Error analysis theo độ dài và khả năng label anomaly

Đoạn mã 6.11. Tạo lỗi theo confidence và accuracy theo quintile độ dài

```
final_mlp.eval(); logits=[]
with torch.no_grad():
    for X_b,_ in test_loader: logits.append(final_mlp(X_b.to(device)).cpu())
logits=torch.cat(logits); score=torch.softmax(logits,1)[:,1].numpy();
pred=logits.argmax(1).numpy(); fig,axes=plt.subplots(1,2,figsize=(12,5));
metrics.ConfusionMatrixDisplay.from_predictions(y_test,pred,display_labels=["Negative","Positiv
e"],cmap="Blues",ax=axes[0]); fpr,tpr,_=metrics.roc_curve(y_test,score);
axes[1].plot(fpr,tpr,label=f"AUC={roc_auc_score(y_test,score):.3f}");
axes[1].plot([0,1],[0,1],"--"); axes[1].legend(); axes[1].set(title="ROC final
MLP",xlabel="FPR",ylabel="TPR"); plt.tight_layout();
save_figure(FIGURE_DIR/"final_confusion_roc.png",dpi=160,bbox_inches="tight"); plt.show()
errors=test_df[["text","label"]].copy(); errors["prediction"]=pred;
errors["positive_score"]=score; errors["review_length"]=errors.text.str.len();
display(errors[errors.label.ne(errors.prediction)].sort_values("positive_score").head(10));
display(errors.assign(correct=errors.label.eq(errors.prediction)).groupby(pd.qcut(errors.review
_length,5,duplicates="drop"),observed=True).correct.agg(["count","mean"]))
```

Bảng 6.5. Xu hướng accuracy theo độ dài review

Quintile độ dài	Accuracy	Xu hướng
Ngắn nhất	93.759%	Cao nhất
Q2	xấp xỉ giữa hai đầu	Giảm nhẹ
Q3	xấp xỉ giữa hai đầu	Giảm nhẹ
Q4	xấp xỉ giữa hai đầu	Giảm nhẹ
Dài nhất	92.028%	Thấp nhất

Phân tích kết quả: accuracy giảm từ khoảng 93,759% ở nhóm ngắn nhất xuống 92,028% ở nhóm dài nhất. Review dài có thể chứa nhiều chủ đề, đảo chiều cảm xúc hoặc ngữ cảnh mà unigram/bigram không nắm đủ; đây là giả thuyết phù hợp output, không phải quan hệ nhân quả đã kiểm định. Một số lỗi confidence cao có nhãn Positive nhưng nội dung rất negative như “DO NOT BUY” hoặc “waste of money”. Các trường

hợp này gợi ý khả năng label noise/source anomaly, không chứng minh chắc chắn dataset gán nhãn sai.

Kết luận task: hidden representation và final MLP đều nhỉnh hơn nhẹ, nhưng LinearSVC vẫn cực kỳ cạnh tranh. Modeling subset cân bằng, TF-IDF thiếu ngữ cảnh dài và optional seed stability chưa chạy; vì vậy không nên khái quát mức chênh lệch 0,0007 F1 ra mọi phân phối review.

CHƯƠNG 7. SO SÁNH VÀ THẢO LUẬN TỔNG HỢP

7.1. Ba mức độ phức tạp của biểu diễn

Ba task tạo một phổ rõ từ dữ liệu bảng ít chiều, qua dữ liệu bảng hỗn hợp, đến văn bản thưa rất nhiều chiều. Dataset size không quyết định parameter count: Diabetes có 177.576 train nhưng MLP chỉ 3.554 tham số; Comments có 800.000 train và first layer 20.000×64 làm mạng vượt 1,28 triệu tham số. House có d=61 nên Baseline chỉ 4.033 tham số dù sample 280.000 train.

Bảng 7.1. Tổng hợp quy mô, final MLP và finding chính

Task	Modeling size	d	Final MLP	Params	Best epoch	Train (s)	Best traditional	Finding chính
Diabetes	177,576/38,052/38,052	21	21→64→32→2	3,554	15	133.97	Random Forest	RF thắng F1; MLP thắng Recall/AUC
House	280K/60K/60K	61	61→64→1	4,033	26	311.47	Random Forest	RF thắng rõ; tail/outlier chi phối
Comments	800K/100K/200K	20,000	20000→64→32→2	1,282,210	2	1,070.94	LinearSVC	MLP hơn 0.000700 F1, cost cao

Phân tích kết quả: thời gian không tăng chỉ theo số dòng. Comments bị chi phối bởi phép nhân first layer cực rộng và validation trên 100K mẫu mỗi epoch; House bị chi phối bởi nhiều epoch cùng inverse/evaluation và RF baseline; Diabetes nhỏ cả d lẫn parameter. Chi phí end-to-end còn gồm vectorization/sampling, không chỉ parameter count.

7.2. Độ sâu và learning rate

Bảng 7.2. Ảnh hưởng của Deeper MLP và learning rate

Task	Baseline	Deeper 1e-3	Deeper 3e-4	Kết luận
Diabetes (F1)	0.440798	0.445066	0.439905	Deeper lợi nhỏ; LR nhỏ không lợi
House (RMSE)	892,011	895,948	896,858	Baseline tốt nhất
Comments (F1)	0.931036	0.931553	0.929574	Deeper lợi rất nhỏ; LR nhỏ kém

Không có bằng chứng rằng thêm một hidden layer cải thiện nhất quán: nó giúp nhẹ ở hai classification task nhưng làm House RMSE xấu đi. Learning rate 3×10^{-4} không thắng ở bất kỳ task nào và thường cần best epoch muộn hơn. Đây không phải kết luận

phổ quát về depth/LR; nó chỉ đúng trong phạm vi kiến trúc, seed, optimizer và epoch budget đã thử.

7.3. Learned representation có thực sự mang lại lợi ích?

Bảng 7.3. Prepared input so với frozen hidden Linear Probe

Task	Metric chính prepared	Metric chính hidden	Thay đổi	Final practical result
Diabetes	F1 0.440033; AUC 0.821494	F1 0.440833; AUC 0.828577	+0.000800 F1; +0.007083 AUC	RF F1 vẫn cao hơn MLP
House	RMSE 2,832,095; R^2 −6.0606	RMSE 890,360; R^2 0.3022	Cải thiện rất rõ	RF/Tree vẫn tốt hơn final MLP
Comments	F1 0.929082; AUC 0.979052	F1 0.931024; AUC 0.980468	+0.001942 F1; +0.001416 AUC	MLP thắng LinearSVC rất nhỏ

Trả lời câu hỏi trọng tâm — Có, learned representation cải thiện cấu trúc hoặc linear decodability trong cả ba task, mạnh nhất ở House và nhẹ ở hai classification task. Tuy nhiên lợi ích representation không bảo đảm Neural Network vượt traditional ML về practical performance: RF thắng Diabetes F1 và House regression; MLP Comments chỉ nhỉnh LinearSVC rất nhỏ với chi phí lớn.

7.4. Metric, threshold và mục tiêu sử dụng

Diabetes minh họa việc lựa chọn metric gắn với chi phí sai lầm: MLP có Recall/AUC cao hơn nhưng false positive nhiều; RF cân bằng F1 tốt hơn. Comments cân bằng class nên Accuracy và F1 đồng thuận hơn, song gain 0,0007 cần đặt cạnh 25,4× thời gian. House cho thấy squared metric đặc biệt nhạy đuôi: một prediction MLP bị clip đủ làm RMSE/R^2 đổi mạnh. Vì vậy báo cáo không gom ba task vào một "điểm thắng" duy nhất.

7.5. Interpretability và khả năng triển khai

Linear/Tree cho phép đọc coefficient, split hoặc feature importance trực tiếp hơn; MLP phân tán thông tin qua nhiều weight nên cần probe và projection. Saved artifact đủ cho inference kỹ thuật — state_dict cùng preprocessor/vectorizer — nhưng production deployment còn cần version schema, threshold policy, monitoring drift và resource budget. Đặc biệt, House phải log cả raw prediction audit và safeguard để tránh che extreme extrapolation.

CHƯƠNG 8. HẠN CHẾ VÀ ĐE DỌA TỚI TÍNH HỢP LỆ

8.1. Hạn chế chung

- Chỉ một random seed và một split; optional seed-stability không chạy, nên chưa có confidence interval hay độ biến thiên giữa seed.
- Không gian kiến trúc hẹp: một hoặc hai hidden layer, width 64/32; chưa thử dropout, weight decay, scheduler, normalization hay tuning threshold.
- Chỉ hai learning rate của Adam và epoch budget hữu hạn; kết luận hyperparameter mang tính cục bộ.
- Toàn bộ chạy CPU; timing hữu ích để so sánh trong môi trường này nhưng không ngoại suy trực tiếp sang GPU/hardware khác.
- PCA/SVD 2D mất thông tin; Linear Probe đo decodability trên validation, không chứng minh nhân quả hoặc robustness ngoài phân phối.

8.2. Diabetes

BRFSS là dữ liệu quan sát và phần lớn tự báo cáo, có thể chứa recall bias, measurement error và confounding. Class imbalance khiến threshold và class weight ảnh hưởng mạnh Precision/Recall. Các dòng giống nhau không được xác định là lỗi. Report không đánh giá calibration, subgroup fairness hay temporal/geographical shift; do đó kết quả không nên dùng như chẩn đoán y tế.

8.3. House Price

Chỉ 400K trong 2,23M dòng và sáu feature được dùng; thiếu location chi tiết, thời điểm giao dịch, chất lượng nhà và nhiều yếu tố thị trường. Missing values, heavy tail và record cực đoan làm metric không ổn định. Train-range clipping là post-processing/domain safeguard có thể tác động metric; raw/safe cùng được báo cáo nhưng chưa thay thế một mô hình robust tail hoặc quy trình data validation tốt hơn.

8.4. Comments

Modeling subset cân bằng nhân tạo không phản ánh chắc chắn class prior thực tế. TF-IDF unigram/bigram thiếu ngữ cảnh dài, mỉa mai và composition. Không thử pretrained contextual encoder. Một số cặp label–text gợi ý khả năng source/label anomaly, nhưng chưa audit nguồn để kết luận. Chi phí MLP lớn và bộ nhớ batch dense là rào cản mở rộng width/vocabulary.

8.5. Hướng cải thiện

Nghiên cứu tiếp theo nên chạy nhiều seed, bootstrap confidence interval, tuning threshold theo cost, calibration classification và nested/robust validation. House cần kiểm tra schema/outlier theo domain, thêm feature location/time, loss robust hoặc mô hình phân vị; safeguard phải tiếp tục audit. Comments có thể thử hashing/feature selection, sparse-aware neural layer hoặc contextual encoder trong compute budget được ghi rõ. Mọi cải thiện phải giữ test đóng cho đến sau selection.

CHƯƠNG 9. KẾT LUẬN

Assignment 03 đã hoàn thành ba bộ controlled experiments MLP trên ba dạng input: tabular ít chiều, mixed tabular sau encoding, và sparse text rất nhiều chiều. Code thể hiện đầy đủ forward propagation, CrossEntropy/MSE loss, loss.backward(), Adam, mini-batch, model.train()/eval(), no_grad và early stopping. Final model của mỗi task được chọn duy nhất trên validation trước khi đánh giá held-out test.

Kết quả không ủng hộ một thông điệp đơn giản rằng mạng sâu hơn hoặc learning rate nhỏ hơn luôn tốt hơn. Deeper MLP chỉ cải thiện nhẹ classification và làm House RMSE xấu hơn; 3×10^{-4} không thắng ở cả ba task. Traditional ML vẫn cạnh tranh mạnh: RF thắng F1 Diabetes và thắng rõ House; LinearSVC gần ngang MLP Comments với chi phí thấp hơn nhiều.

PCA/SVD và Linear Probe cho thấy hidden representation có thể cải thiện linear decodability: nhẹ ở Diabetes/Comments và rất rõ so với prepared linear model ở House. Tuy vậy representation tốt hơn không đồng nghĩa practical superiority. Computational cost, outlier sensitivity, class trade-off và tính minh bạch đều phải tham gia quyết định mô hình.

Finding khoa học quan trọng nhất là sự tách biệt giữa chất lượng biểu diễn và chất lượng hệ thống cuối. Neural network có thể học feature map hữu ích mà vẫn thua một ensemble phù hợp với dữ liệu; hoặc thắng metric rất nhỏ nhưng không thắng cost-performance. Kết luận này trung thực với output đã chạy và tạo nền tảng tốt cho các thí nghiệm có uncertainty estimate trong tương lai.

CHƯƠNG 10. TÀI LIỆU THAM KHẢO

Các mục dưới đây chỉ ghi những nguồn/phần mềm và nguồn dữ liệu được nêu trong project; không bổ sung tác giả hoặc năm khi metadata bibliographic không có trong notebook.

[1] PyTorch documentation — torch.nn, autograd, optim.Adam và DataLoader.

[2] scikit-learn documentation — preprocessing, traditional estimators, metrics, PCA và TruncatedSVD.

[3] CDC BRFSS 2015 — Diabetes Health Indicators dataset, nguồn được sử dụng trong notebook Diabetes.

[4] USA Real Estate dataset — nguồn dữ liệu price/property được cấu hình trong notebook House Price.

[5] Amazon Review Polarity dataset — train/test CSV được cấu hình trong notebook Comments.

[6] NumPy, pandas, SciPy, Matplotlib và joblib documentation.

[7] Ba executed notebooks trong notebooks/assignment03 và 12 CSV kết quả trong results/assignment03, nguồn thực nghiệm chính của báo cáo.

PHỤ LỤC A. SOURCE CODE VÀ ARTIFACT

A.1. Standalone source

Bảng 11.1. Các module standalone của Assignment 03

File	Vai trò
src/assignment03/common.py	Tiện ích seed, MLP, metric/training dùng chung
src/assignment03/diabetes.py	Pipeline Diabetes classification
src/assignment03/house_price.py	Pipeline House regression và inverse audit
src/assignment03/comments.py	Pipeline sparse TF-IDF và sentiment MLP

Các module phản ánh logic notebook dưới dạng callable source. Notebook vẫn là nguồn code excerpt chính vì chứa output, narrative và decision trail của lần chạy FULL.

A.2. Saved models, preprocessors và vectorizer

Bảng 11.2. Artifact phục vụ tái sử dụng

Task	Model	Biến đổi đầu vào
Diabetes	models/assignment03/diabetes/final_mlp.pt	preprocessor.joblib
House	models/assignment03/house_price/final_mlp.pt	preprocessor.joblib
Comments	models/assignment03/comments/final_mlp.pt	tfidf_vectorizer.joblib

A.3. CSV kết quả và figure

Mỗi task có bốn CSV: baseline_validation, mlp_experiments, model_comparison và training_history. Mỗi task có bốn PNG: EDA, training curves, final evaluation/error và representation. Tổng cộng 12 CSV và 12 figure; tất cả figure đã được chèn đúng chương trong báo cáo.

A.4. Checklist lựa chọn và đánh giá

- Diabetes: Deeper 1e-3 chọn theo validation F1/loss; test F1=0.449114, AUC=0.829583.
- House: Baseline chọn theo validation RMSE/MAE; test báo cả raw và safeguarded, n_clipped=1.

- Comments: Deeper 1e-3 chọn theo validation F1/loss; official test F1=0.929900, AUC=0.980112.
- Không dùng test để chọn clipping bound; House bound chỉ lấy min/max y_train_model.
- Optional diagnostics/seed stability không chạy và không được trình bày như kết quả đã có.